

通过 NPUs 进行快速设备上 LLM 推理

徐大亮 北京大学高等科学技术教育部重点实验室；中国北京大学 SCS xudaliang@pku.edu.cn

张浩 北京邮电大学，中国 hao_zhang@bupt.edu.cn

杨黎明，北京大学高等科学技术教育部重点实验室；北京大学计算机科学系 yangliming2018@pku.edu.cn

刘瑞琪 北京大学高等科学技术教育部重点实验室；中国北京大学 SCS rich@stu.pku.edu.cn

黄刚 北大HCST教育部重点实验室；SCS，北京大学，中国数据空间技术与系统国家重点实验室，中国 hg@pku.edu.cn

徐孟伟* 北京邮电大学，中国 mwx@bupt.edu.cn

刘轩哲* HCST（北京大学）教育部重点实验室；中国北京大学 SCS liuxuanzhe@pku.edu.cn

抽象的

在日益增长的隐私问题和移动规模模型的进步的推动下，大型语言模型 (LLM) 的设备端推理引起了人们的极大兴趣。然而，即使是移动大小的 LLM（例如 Gemma-2B）也会遇到不可接受的高推理延迟，通常会受到屏幕 UI 理解等任务中预填充阶段的瓶颈。

我们推出了 llm.npu，这是第一个利用设备上神经处理单元 (NPU) 卸载来减少预填充延迟的 LLM 推理系统。llm.npu 通过在三个层面上重构提示和模型来提高 NPU 卸载效率：（1）在提示层面，它将可变长度提示划分为多个固定大小的块，同时保持数据依赖性；（2）在张量级别，它识别并提取显著的异常值，以最小的开销在 CPU/GPU 上并行运行；（3）在块级别，它根据硬件亲和性和对准确性的敏感性，以无序方式将 Transformer 块调度到 CPU/GPU 和 NPU。与竞争基准相比，llm.npu 的预填充速度提高了 22.4×，能源节省了 30.7×

平均而言，在端到端的实际应用中加速高达 32.8×。llm.npu 首次实现了十亿规模模型每秒超过 1,000 个代币的预填充。

CCS 概念：• 以人为本的计算 → 无处不在的移动计算；
• 计算机系统组织 → 异构（混合）系统；
方法 → 自然语言处理。

关键词：移动计算、大语言模型、NPU

ACM 参考格式：

徐大良、张浩、杨黎明、刘瑞琪、黄刚、徐孟伟* 和刘轩哲*。2025. 使用 NPU 进行快速设备上 LLM 推理。第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集，第 1 卷 (ASPLOS '25)，2025 年 3 月 30 日至 4 月 3 日，荷兰鹿特丹。ACM，美国纽约州纽约市，18 页。https://doi.org/10.1145/3669940.3707239

1 简介

随着隐私问题的日益严重[5]，人们对在移动设备上本地运行大型语言模型 (LLM) 越来越感兴趣，称为设备上 LLM [99,100,102]，例如 Apple Intelligence [2] 和 Android AI Core [1]。与此同时，移动大小的语言模型（1B-10B 参数）的进步，例如 Qwen2-1.5B 和 Phi3-3.7B，已经证明了它们的性能与 GPT-3 等更大的模型相当，尽管参数数量减少了 [24,26,27,60]。这一进展使得设备上语言模型的部署变得可行。在不泄露私人数据的情况下，设备上的 LLM 推理可以催化新颖的移动应用程序 [90]，例如 UI 任务自动化 [55]（例如，将用户的语言命令翻译为 UI 操作，例如“将未读电子邮件转发给 Alice”）和自动消息回复 [20]。

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0698-1/25/03...\$15.00

https://doi.org/10.1145/3669940.3707239

然而，高推理延迟仍然是实际设备上 LLM 的重大障碍。为了完成 UI 任务，LLM 需要摄取屏幕视图层次结构（通常为 600-800 个标记 [22, 82]）以逐步生成相应的 UI 操作 [81]。正如 §2.1 中所示，对于 Qwen1.5-1.8B [27]，每个此类步骤需要 8.1 秒，因此完成 5 步 UI 任务需要 40 秒以上。同样，Gemma-2B 模型根据历史电子邮件数据（具有 1500 个令牌）模仿用户的语气，需要 26.7 秒来自动回复电子邮件。深入研究这些任务，我们发现提示处理（预填充阶段）通常主导端到端推理延迟，例如 UI 自动化任务的 94.4% - 98.8%。这是因为设备上的 LLM 任务通常涉及长上下文理解以处理个性化任务。不幸的是，现有的研究工作主要集中在加快文本生成速度（解码阶段），例如激活稀疏性 [74, 94] 和推测解码 [50, 61, 95]。因此，这项工作的主要目标是提高设备上 LLM 的预填充速度。

LLM 预填充受计算限制 [69, 83, 106]；然而，移动 CPU 和 GPU 的并行计算能力有限 [40, 97]。相反，我们受到神经处理单元（NPU）在现代移动设备中无处不在的关键机遇的激励，例如高通 Hexagon NPU 和 Google Edge TPU。这些移动 NPU 在整数向量运算方面非常高效，计算能力高达 73 TOPS [17]。在 CNN 上，它们相对于移动 CPU/GPU 的改进被证明分别高达 $18\times/4\times$ [86]。与 CPU/GPU 相比，移动 NPU 也更节能，并且工作负载竞争更少。

令人惊讶的是，尽管有这些承诺的优势，但目前还没有支持 COTS 移动 NPU 上的 LLM 推理的系统。事实上，我们的初步工作表明，由于以下挑战，直接使用移动 NPU 进行 LLM 推理并不能提供性能优势。

- 可变长度提示的准备成本高昂。移动 NPU 通常仅支持静态形状的推理，而 LLM 提示长度是动态的（具有最大上下文长度）。在移动设备上，为每个不同大小的提示在 NPU 上重新准备和优化 LLM 执行图的成本很高（例如，Gemma-2B 模型需要 11 秒）。另一方面，简单地将 LLM 请求填充到相同的最大上下文长度会浪费宝贵的计算资源。

- LLM 量化算法与移动 NPU 设计之间的不匹配。由于异常激活的存在 [33, 84]，最先进的 LLM 量化方法通常使用每组量化来保持高精度。它将原始激活和权重张量划分为多个具有独立量化尺度的组，以避免异常值对其他组的影响。然而，我们的研究表明移动 NPU 无法直接执行每组 MatMul（表 2）。相反，他们必须将 MatMul 拆分为多个组大小的子张量 MatMuls，然后使用浮点和来减少子张量中间结果

手术。此过程会影响 NPU 效率并产生高达 $10.7\times$ 的性能开销。

- 浮点 (FP) 运算无法消除。移动 NPU 通常提供显著的基于整数的 Mat-Mul 加速，但在 FP 运算方面较弱。然而，LLM 很难以最小的精度损失量化为纯整数执行。现有的量化 LLM 仍然依赖于像 LayerNorm 和 Attention 这样的浮点运算符。将这些 FP 算子调度到 NPU 之外可以轻松地增加推理关键路径。

这项工作提出了 llm.npu，这是第一个具有高效设备上 NPU 卸载功能的 LLM 推理系统。llm.npu 的主要设计目标是减少预填充延迟和能耗。它针对的是 LLM 的主流仅解码器 Transformer 架构（例如 LLaMA、GPT 等）。关键思想是最大限度地提高移动 NPU 上的预填充执行速度，以加速整数计算，同时在 CPU/GPU 上保留必要的浮点运算以保持准确性。为了克服上述挑战并提高 NPU 卸载效率，llm.npu 在三个层面上重新构建了提示和模型：（1）在提示级别：llm.npu 将可变长度提示划分为多个固定大小的块，同时保持数据依赖性；（2）在张量级别：llm.npu 识别并提取显著异常值以在 CPU/GPU 上运行；（3）在块级别：llm.npu 根据硬件亲和性和对准确性的敏感性将 Transformer 块调度到 CPU/GPU 和 NPU。相应的新技术详述如下：

- 块共享图 (§3.2) llm.npu 将可变长度提示拆分为带有预先构建的子图的固定大小的“块”，从而减少了图准备时间。然后，在具有块级因果依赖性的那些图上执行每个可变长度提示。这种方法利用了令牌生成仅依赖于仅解码器 LLM 中的先前令牌的见解。然而，同时加载多个预构建的块图会产生大量内存开销，例如比 LLM 权重多 $2-4\times$ 。为了解决这个问题，llm.npu 进一步识别与提示大小无关的运算符（例如 FFN），并在每个块执行中共享它们，从而将内存开销减少多达 $4\times$ 。

- 影子离群值执行 (§3.3) 在不影响 NPU 效率的情况下解决了激活离群值问题。在不更改 NPU 上每个张量 MatMul 的情况下，llm.npu 将激活离群值通道提取到紧凑张量中，并在 CPU/GPU 上并行执行，因为离群值非常稀疏（占总通道的 0.1%–0.3%）。由于 CPU 内存中重复的 MatMul 权重以及 CPU/GPU 和 NPU 之间的同步开销，该设计进一步引发了内存占用增加的问题。因此，基于异常值更有可能出现在一小部分通道位置的观察结果，llm.npu 通过仅将那些“热通道”权重保留在内存中来优化内存使用，并根据需要从磁盘检索其他权重。LLM.npu

还通过测量异常值的重要性来修剪层级别的不重要异常值，以减少同步开销。

- 无序子图执行 (§3.4) llm.npu 以一个关键见解为指导，即可以以无序方式调度多个子图，而无需严格遵循原始提示中的块顺序。这显着扩大了 llm.npu 的调度空间，以最大限度地减少 CPU/GPU 浮动操作带来的执行气泡。鉴于寻找最佳乱序执行顺序是一个 NP 难题，llm.npu 采用了微秒级在线调度算法。该算法基于观察到 NPU 的工作负载较重并且构成关键路径。因此，在选择执行哪个子图时，llm.npu 会优先考虑那些对减少 NPU 停顿影响更显著的子图，而不是仅仅关注子图的执行延迟。值得注意的是，llm.npu 的调度算法并没有最大化并行处理能力。相反，llm.npu 的目标是最大限度地提高 NPU 的利用率，同时最大限度地减少 CPU/GPU 工作负载的影响。

实施和评估。我们在 MLLM [23] 和 QNN [25] 之上实现了 llm.npu，其中包含 10K 行 C/C++ 和汇编代码。我们使用五个移动大小的 LLM (Qwen1.5-1.8B [27]、Gemma-2B [9]、phi2-2.7B [16]、Llama-2-7B [11] 和 Mistral-7B [14])、四个 LLM 基准测试和两个移动设备 (小米 14 和 Redmi K60 Pro) 评估了 llm.npu。我们将 llm.npu 与五个竞争基准进行了比较，包括三个工业开源引擎 (llama.cpp [59]、TFLite [57]、MNN [49]) 和两个最先进的研究原型 (MLC-LLM [75] 和 PowerInfer-v2 [94])。实验表明，llm.npu 在预填充延迟和能耗方面显着且一致地优于所有基线，同时保持推理精度 (与 FP16 相比 < 损失 1%)。它比 CPU 上的基准快 7.3x–18.4x，在 GPU 上比基准快 1.3x–43.6x，提示长度为 1024。它还实现了 1.9x–59.5x 能耗降低。据我们所知，llm.npu 是第一个在 CO TS 移动设备上为数亿规模的 LLM 实现 >1000 个令牌/秒预填充速度的系统。在端到端的实际应用中，与基线相比，llm.npu 将推理延迟 (预填充+解码) 减少了 1.4x–32.8x。

贡献总结如下：

- 我们深入研究了使用移动 NPU 加速 LLM 预填充的挑战和机遇。
- 我们提出了第一个具有高效移动 NPU 卸载功能的 LLM 推理引擎，具有三种新颖的技术：块共享图、影子异常值执行和乱序子图执行。
- 我们在 llm.npu 上进行了全面的实验，证明了其优于竞争基线的性能。llm.npu 的代码可在 <https://github.com/UbiquitousLearning/mlm> 获取。

2 背景

2.1 设备上 LLM 推理分析

由于增强的隐私保护，设备上的**大语言模型**越来越多地用于尖端场景，例如 Apple Intelligence [2]、UI 自动化 [81] 和自动电子邮件回复 [20]。为了支持这些应用程序，人们开发了许多轻量级 LLM，如表 1 所示。例如，根据 DroidTask [22, 82]、LongBench 数据集的评估，llama.cpp 上的 Qwen1.5-1.8B 模型在使用移动 CPU 时，单步 UI 自动化延迟为 8.1 秒，自动电子邮件回复延迟为 21.7 秒，聊天摘要延迟为 8.4 秒。[28] 和 Persona-Chat [48]，这对于现实世界的部署来说是不切实际的。然而，它们的推理延迟仍然是一个重大挑战。

为了详细探讨这个问题，我们分解了上面讨论的推理过程。结果如图 1 所示。令人惊讶的是，我们发现与已广泛研究的解码阶段相比，预填充阶段通常是典型移动应用中的瓶颈。例如，预填充阶段占 UI 任务自动化、聊天摘要和上下文感知生成任务的移动 CPU 总延迟的 8.3%–98.8%。即使使用移动 GPU，预填充阶段仍然占主导地位，占总延迟的 54.2% 至 91.7%。随着提示长度的增加，预填充阶段占总推理时间的比例也会增加。

造成这种情况的因素有几个：(1) 移动 CPU/GPU 缺乏云 GPU 的并行能力 [40]，主要设计用于处理应用程序逻辑或渲染任务。(2) 移动**大语言模型**通常需要很长的提示才能进行个性化、上下文感知的生成。例如，自动电子邮件回复通常需要大量的用户数据，例如历史电子邮件、日程安排和位置信息，在 LongBench 数据集中总计有 1,168–1,835 个令牌 [28]。同样，从事 UI 自动化的设备上**大语言模型**处理大量的 UI 注释标记 (XML 或 HTML) 和用户命令，在 DroidTask 数据集中有 505–827 个标记 [22, 82]。然而，这些应用程序的输出长度通常很短，LongBench 数据集平均为 7.9 个标记，DroidTask 数据集平均为 3.5 个标记。在聊天摘要的上下文中，例如 Persona-Chat 数据集 [48]，提示和输出长度相对平衡，输入大小在 488–584 个令牌之间，平均输出为 44 个令牌。(3) 移动法学硕士现在支持长上下文窗口。例如，Qwen2-1.5B 等最新模型可以容纳最多 32K 个标记的上下文窗口，如表 1 所示。

2.2 机遇：移动 NPU

为了优化预填充延迟，llm.npu 利用了一个关键机会：现代移动 SoC 普遍包含移动

表 1. 移动规模 LLM 的最大上下文长度。

Model	Max Context	Year	Model	Max Context	Year
Opt-1.3B	2K	2022.5	TinyLLaMA-1.1B	2K	2023.9
StableLLM-3B	4K	2023.10	phi-2-2.7B	2K	2023.12
Gemma-2B	8K	2024.2	Qwen1.5-1.8B	32K	2024.2
Phi3-mini-3.8B	128K	2024.5	Qwen2-1.5B	32K	2024.6

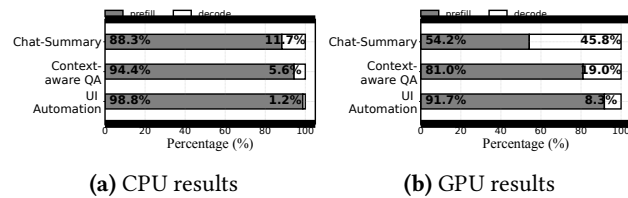


图 1. UI 自动化、上下文感知 QA 和聊天摘要的端到端推理延迟细分。CPU 评估使用 llama.cpp 作为设备端推理引擎，GPU 评估使用 TFLite 作为模拟引擎。

表2. 主流厂商提供的知名移动NPU规格。

Vendor	Latest NPU	SDK	Open	Group	INT8 Perf.
Qualcomm	Hexagon NPU [17]	QNN [25]	×	×	73 TOPS
Google	Edge TPU [19]	Edge TPU API [8]	×	×	4 TOPS
MediaTek	MediaTek APU 790 [12]	NeuroPilot [15]	×	N/A	60 TOPS
Huawei	Ascend NPU [7]	HiAI [10]	×	×	16 TOPS

“开放”：开源？；“Group”：支持每组量化 MatMul？ “N/A”：没有可供公众使用的文件；“INT8 Perf.”：Int8 性能。

神经处理单元（NPU）非常适合整数运算，例如基于 INT8 的矩阵乘法。表2总结了主流厂商提供的知名移动NPU的规格。例如，高通的移动 SoC 采用 Hexagon NPU，每秒可实现高达 73 万亿次 INT8 运算。根据 AI-Benchmark [18]，小米 14 中的 Hexagon NPU 只需 0.6 ms 即可推理出 MobileNet-V2 模型，比移动 CPU 快 23×，比移动 GPU 快 3.2×。

移动NPU架构。移动 NPU 通过单指令多数据 (SIMD) 架构提供显著的性能优势。例如，Hexagon NPU 支持 10 24 位 INT8 向量运算，允许多个 SIMD 指令并行执行。然而，与移动 GPU 相比，它们的浮点计算能力相对较弱。移动 NPU 的时钟频率在 500 至 750 MHz 之间，比移动 CPU 和 GPU 更节能。此外，与具有独立物理内存的云GPU不同，移动NPU集成在移动SoC中，与移动CPU共享相同的物理内存，从而无需在NPU执行期间进行内存复制。

微观基准比较。为了评估 INT8 MatMul 在移动 NPU 上的性能，我们使用移动 LLM 中常用的 MatMul 尺寸在 Redmi K70 Pro 上进行了初步实验，总结如下：

表 3. Redmi K70 Pro 上各种 MatMul 大小的执行延迟（毫秒）。

Matrix A	Matrix B	NPU INT8	CPU INT8	GPU FP16	NPU FP16
64*2048	2048*2048	0.9	4.2 (4.6×)	1.7 (1.9×)	252 (193×)
64*2048	2048*8192	1.5	6.8 (4.5×)	4.8 (3.2×)	986 (657×)
64*2048	2048*11008	2.0	11.6 (5.8×)	6.9 (3.5×)	1207 (603×)
32*4096	4096*4096	1.7	7.5 (4.4×)	3.1 (1.8×)	1054 (620×)
32*4096	4096*8192	2.9	13.1 (4.5×)	7.7 (2.6×)	2009 (692×)
32*4096	4096*11008	4.1	19.6 (4.8×)	10.4 (2.5×)	3112 (759×)

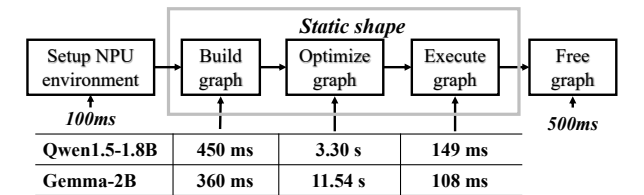


图 2. 在移动 NPU 上执行 DNN 的工作流程，以及 QNN 上每个过程的延迟 [25]。

表 3. 与 CPU INT8 相比，移动 NPU 上的 INT8 MatMul 实现了 4.5–5.8× 加速，比 GPU FP16 实现了 1.8–3.5× 改进。计算工作量越大，性能提升就越明显。然而，在移动 NPU 上执行 FP16 MatMul 导致性能比 CPU INT8 慢 1 59×。这些结果与移动 NPU 的 INT8 SIMD 架构一致，证实移动 NPU 最适合加速 INT8 矩阵乘法。

移动 NPU 上的 DNN 执行工作流程。在移动NPU上执行DNN涉及配置NPU环境、创建计算图、优化图、执行图和释放图，如图2所示。通常，创建和优化计算图是最耗时的，前者包括将模型转换为NPU所需的中间表示和内存分配，需要300-500ms，而后者包括调整内存布局、执行顺序和算子融合的优化，需要花费很多时间。秒。此外，NPU SDK 的闭源性质限制了 LLM 的进一步适配。

2.3 LLM 和移动 NPU 之间的差距

鉴于其固有的优势，我们惊讶地发现现有的 DNN 引擎都不支持移动 NPU 上的 LLM 加速。然后，我们深入研究根本原因，发现现有移动 NPU 设计与 LLM 推理流程之间存在巨大差距。

- LLM 预填充阶段依赖于可变长度的提示，导致在构建和编译 NPU 图上花费过多的时间。如图 2 所示，计算图在移动 NPU 上执行之前，必须对其进行构建和优化，该过程需要数十秒。例如，为 Gemma 2B 构建图表

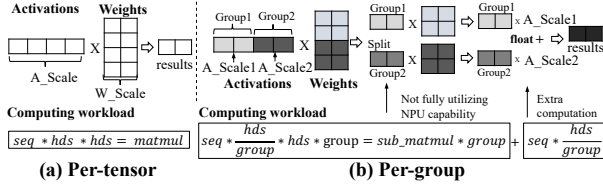


图 3. 每张量量化 MatMul 和每组量化 MatMul。seq、hds、group 分别表示序列长度、隐藏大小和组数。

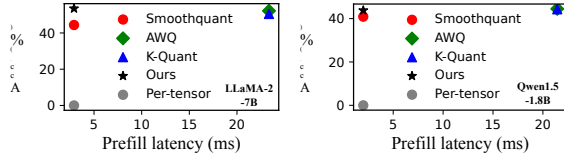


图 4. 使用 Qualcomm QNN 框架的小米 14 上不同量化算法之间 HelloSwag 数据集的预填充延迟和准确性。

使用 QNN 框架的模型需要 360 毫秒，图优化需要 11.54 秒。与 CNN 模型不同，CNN 模型构建和优化一次，并且可以使用相同的输入形状多次执行，LLM 预填充阶段必须处理可变长度的提示，从而需要为每个推理重建和重新优化计算图。因此，在这种情况下使用移动 NPU 不会带来任何性能优势，而且速度通常比使用 CPU 慢。

● 激活异常值的存在使得 LLM 难以在整个张量级别上进行量化，而更细粒度的组级别量化会阻碍 NPU 效率。我们的初步实验如图 4 所示，表明两种流行的量化算法（K-Quant [59] 和 AWQ [56]）与每张量量化相比（图 3(a)）会产生 $8.1\times$ – $10.7\times$ 的显著推理开销。这是因为 K-Quant 和 AWQ 等算法使用细粒度的每组量化（图 3(b)）来保持高精度。这些算法将激活和权重分为多个组，每个组都有独立的量化尺度。在 NPU 上，这种方法需要将 MatMul 操作划分为多个子张量 MatMul，这无法充分利用移动 NPU 的能力。此外，它还需要通过浮点加法来聚合中间结果，从而导致额外的浮点计算。SmoothQuant [84] 是个例外，它使用每张量量化，但精度损失显著，例如 LLaMA-2-7B 和 Qwen1.5-1.8B 模型的 HelloSwag 数据集分别下降了 3.9% 和 8.4%。

● 设备上 LLM 推理依赖于浮点运算，与 NPU 的 INT8 加速设计相冲突。图 5 说明了典型的工作流程

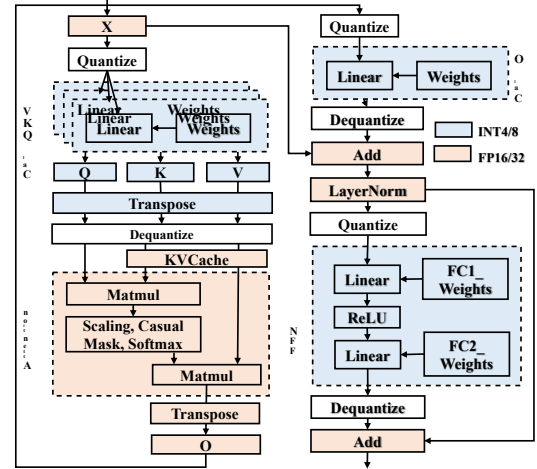


图 5. 量化设备上 LLM 推理的工作流程。蓝色显示的操作是使用 INT4/INT8 格式计算的，而橙色显示的操作是使用浮点数据格式计算的。

表 4. 最先进的量化推理方法中的运算符数据格式。第一行中的每个模块如图 5 所示。

Quantization	Type	Acc.	Cal QKV	Atten.	Cal O	Norm.	FFN
K-Quant [59]	Per-Group	Low	INT8	FP16	INT8	FP16	INT8
GPTQ [35]	Per-Group	High	FP16	FP16	FP16	FP16	FP16
AWQ [56]	Per-Group	High	FP16	FP16	FP16	FP16	FP16
SmoothQuant [84]	Per-tensor	Low	INT8	FP16	INT8	FP16	INT8

“注意”：注意；“规范”：规范化。

量化推理。为了确保推理准确性，只有线性层（以蓝色突出显示）以 INT8/INT4 数据格式执行矩阵乘法。对于这些层，在使用权重执行点积之前，激活 x 被量化为 INT8/INT4。其他操作，例如 Attention 和 LayerNorm（以橙色突出显示），以浮点格式计算。表 4 进一步总结了最先进的量化推理算法中的运算符数据格式。所有这些都依赖于浮动注意力和标准化操作。鉴于浮点运算会极大地降低移动 NPU 的性能，因此这些运算无法在 NPU 上高效执行，且开销较大。

3 llm.npu 设计

3.1 llm.npu 概述

设计目标。llm.npu 旨在通过设备上的 NPU 卸载来减少移动规模 LLM 的预填充延迟和能耗。它支持设备上的各种移动大小的 LLM，并且可以作为 LLM 即系统服务的一部分集成到移动操作系统或移动应用程序服务中 [99, 102]。工作流程。图 6 说明了 llm.npu 的工作流程。llm.npu 的核心思想是最大化其在移动 NPU 上的执行，以实现整数运算加速；同时保持

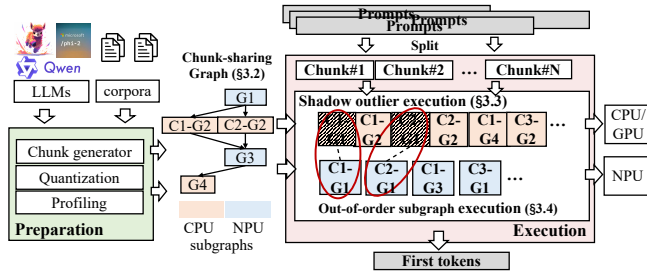


图 6. llm.npu 的工作流程。

CPU/GPU 上必要的浮点运算，以免影响精度。为了实现更高效的 NPU 卸载，llm.npu 通过以下方式重新构建提示和模型：（1）在提示级别：可变长度提示被缩减为多个固定大小的块，并保留数据依赖性；（2）块级：Transformer 块根据硬件无限性和精度敏感度被调度到 CPU/GPU 和 NPU 中；（3）在张量级别：识别并提取重要的异常值以在 CPU/GPU 上运行。

- 准备阶段。llm.npu 首先使用增强的每张量量化算法将 LLM 量化为 W8A8 格式。量化算法与现有算法不同，因为它过滤掉最不重要的激活异常值，并将其余的提取到独立的、轻量级的算子中，作为原始算子的补充。llm.npu 还生成固定长度的块共享图 (§3.2)，以有效地处理可变长度的提示。

- 执行阶段。当收到提示时，llm.npu 将其分为固定大小的块并按因果关系进行处理。这些块图将根据数据格式分成调度到 CPU/GPU 和 NPU 上的子图，以实现高效执行。为了保持准确性，某些基于 INT8 的线性层在 CPU/GPU 上并行进行稀疏浮点离群值阴影执行，以补偿离群值的量化误差 (§3.3)。为了提高执行效率，llm.npu 明智地以无序方式调度块 (§3.4)。

3.2 块共享图执行

为了解决动态提示长度的挑战，一个直观的解决方案是预先设置一个固定长度的计算图并使用填充[3,67,77]。然而，这种方法缺乏灵活性，并且过多的填充会浪费计算资源。

分块预填充。为了增强灵活性并最大限度地减少可变长度提示的填充，我们认识到在 LLM 中处理长提示相当于按因果关系处理多个分割的子提示或“块”。这是可行的，因为流行的 LLM 使用仅解码器架构，其中第 i 个令牌的结果仅取决于前面的令牌。为此，llm.npu 首先在准备阶段预构建和预优化基于固定长度块的 NPU 计算图。在推理过程中，llm.npu 进行长分割

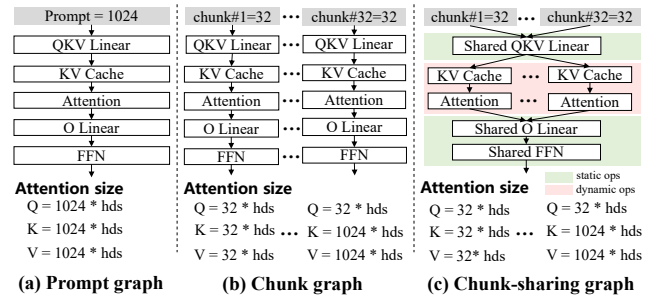


图7.提示图、块图和块共享图的图示。块长度为 32。

提示分成几个块并使用这些预先构建的块图处理它们，如图 7(b) 所示。

然而，单独使用块图是不可扩展的，因为 llm.npu 需要在内存中存储大量不同的块图，从而显著增加内存开销。这是因为不同的块图具有不同大小的注意力运算符。例如，考虑到提示长度为 1024，块长度为 32，第一个块的注意力算子的 QKV 维度大小都是 $32 * hds$ ，而最后一个块的 QKV 维度大小分别是 $32 * hds$ 、 $1024 * hds$ 和 $1024 * hds$ ，如图 7(b) 所示。

块共享图。llm.npu 引入了一个块共享图，如图 7 (c) 所示，基于 LLM 运算符分为两个不同类别的见解：

- （1）静态运算符（绿色），例如 Linear 和 LayerNorm，它们仅依赖于块长度并且可以在不同块之间共享；
- （2）动态算子（红色），例如 Attention，它依赖于块长度和块序列，并且不能在不同块之间共享。因此，llm.npu 根据操作员的可共享性将 LLM 分为几个子图。共享子图构建并优化一次，而非共享子图针对不同块单独构建。在预填充阶段，来自不同块的激活通过相同的静态运算符子图，同时动态选择适当的特定于维度的动态运算符。这种方法显著减少了内存开销并增强了可扩展性，因为大多数动态运算符（例如注意力）不包含权重，只需要激活缓冲区。

我们的实验表明，144 个子图中的 120 个可以在 Qwen 1.5-1.8B 模型中共享，对于提示长度为 1024 和块长度为 256 的情况，内存消耗最多可减少 75% (7.2GB)。

我们还进行了广泛的实验来选择合适的块长度。两种流行的 LLM (Qwen1.5-1.8B 和 Gemma-2B) 在小米 14 设备上的结果如图 8 所示。根据观察，llm.npu 经验性地为小米 14 设备选择了 256 的块长度，这有效地利用了移动设备的能力

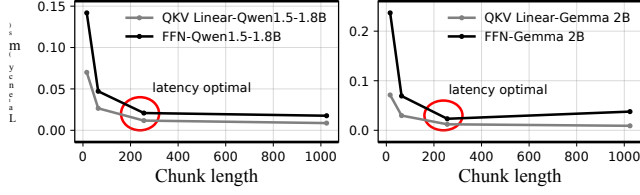


图 8. QKV Linears 和 FFN 在不同块长度下的每个令牌延迟。

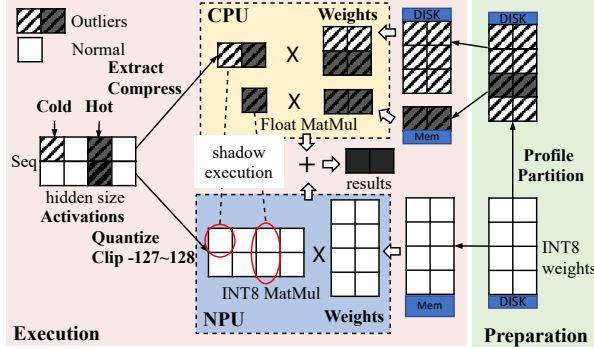


图 9. 影子异常值执行的工作流程。

NPU 同时减少块内填充。实际上，这种分析需要在不同的 NPU 上执行。

3.3 影子异常值执行

为了在不影响 LLM 精度的情况下实现 NPU 友好的每张量激活量化，llm.npu 采用了一种称为阴影异常值执行的新颖方法。如图 9 所示，llm.npu 在运行时将具有异常值的激活通道提取为更紧凑的张量，在 CPU 上执行，然后将其合并回 NPU 上原始算子的结果。该程序可以表述如下：

$$\begin{aligned} \frac{x}{s} \odot w &= \left\{ \min \left[\max \left(\frac{x}{s}, -127 \right), 128 \right] + \left\lfloor \frac{x}{s} / 128 \right\rfloor \times 128 \right\} \odot w \\ &= \min \left[\max \left(\frac{x}{s}, -127 \right), 128 \right] \odot w \quad \text{on NPU} \\ &\quad + \text{extract} \left(\left\lfloor \frac{x}{s} / 128 \right\rfloor \times 128 \right) \odot w \quad \text{on CPU} \end{aligned} \quad (1)$$

其中 x 、 w 、 s 、 $\odot$ 和 extract 分别表示原始浮点激活、INT8 权重、量化比例因子、MatMul 操作以及将激活异常值提取到更紧凑的张量中的函数。具体来说，根据结合律， $\text{MatMul}_s^x \odot w$ 可以等价地分为两部分之和：(1) 尺度内 MatMul 的移动 NPU。llm.npu 首先根据比例因子 s 将 x 量化并舍入到 -127 到 128 的范围。然后，它通过使用权重 w 执行标准 W8A8 每个张量 MatMul 来获得中间结果。(2) 超出规模的用于 MatMul 的移动 CPU/GPU。llm.npu 计算超过 s 的部分值。由于这些异常值很少见，llm.npu

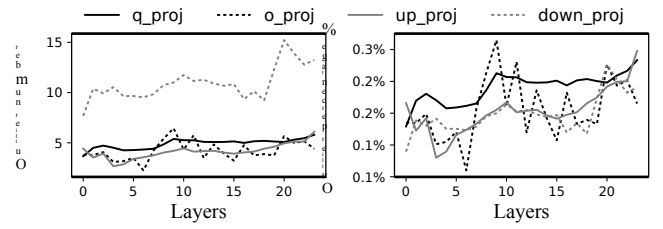


图 10. 使用 wikitext 数据集在 2048 次推理下的 Qwen1.5-1.8B 模型上每层离群值通道的平均数量和百分比。在一次推理过程中，只有不到 0.3% 的通道存在异常值。

从张量中提取这些值，将它们压缩为稠密张量，并使用权重 w 执行 MatMul。

由于异常值非常稀疏（大约 5-15 个通道，仅占总通道的 0.1%-0.3%，如图 10 所示），CPU 上的影子执行速度比 NPU 上原始张量的执行速度快得多，并且其执行时间可以通过重叠完全隐藏。为了进一步最小化这个额外过程的开销，llm.npu 通过离线分析大型语料库来确定异常值阈值（即等式 1 中的 s ），从而可以通过简单地将激活数与该阈值进行比较来识别异常值。阴影异常值执行的设计与任何每张量量化算法兼容，并且 llm.npu 的当前原型基于简单的最大最小对称量化[47]。

虽然影子异常值执行似乎很好地平衡了 NPU 亲和力和 LLM 准确性，但还需要解决两个更关键的问题才能使其实用。首先，虽然移动 SoC 对异构处理器使用统一的存储芯片，但它们使用独立的存储空间。为了启用激活异常值的影子执行，llm.npu 必须在 CPU 内存空间上保留每个 MatMul 权重的另一个副本。这使内存占用增加了近 2 倍。其次，虽然异常值的执行即使在 CPU 上也很快，但 CPU 和 NPU 之间的减少总和的同步仍然需要不小的开销，例如，Qwen1.5-1.8B 上的端到端延迟为 29.7%，能耗为 20.1%。

大多数异常值往往出现在一小部分通道位置中。我们观察到，虽然在处理长提示期间（例如 78%），异常值出现在各种通道位置，但这种外观是高度倾斜的——很少有通道主导异常值的出现。例如，如图 11 所示，在 Qwen1.5-1.8B 和 LLaMA-2-7B 模型的各种输入中，不到 3% 的通道对大多数异常值（超过 80%）有贡献。因此，对于影子异常值执行，llm.npu 仅将那些“热通道”需要使用的张量权重保留在 CPU 内存空间中，如果在这些位置上提取异常值（这种情况很少见），则从磁盘检索其余部分。

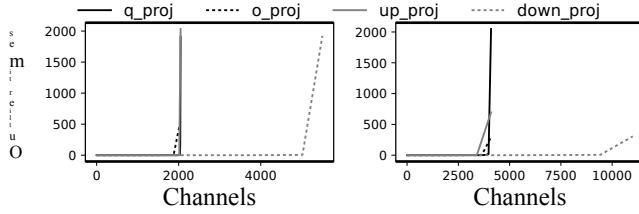


图 11. 使用 wikitext 数据集在 2048 推断下的 Qwen1.5-1.8B 模型上每个通道的异常值时间。不到 3% 的渠道贡献了超过 80% 的异常值。

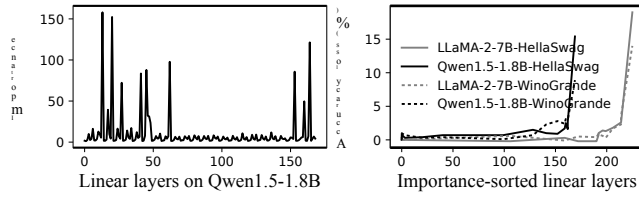


图 12. 左: Qwen1.5-1.8B 模型中不同层的异常重要性。右图: HellaSwag 和 Winograde 数据集上的准确度与修剪层之间的关系。

运行时。请注意，权重检索也可以与原始 MatMul 的 NPU 执行重叠。这种方法将影子执行的内存开销减少了 34.3%，并且延迟可以忽略不计。

大多数异常值都可以被修剪，而不会影响准确性。令人惊讶的是，我们观察到大多数 MatMul 算子上的激活异常值对 LLM 精度并不重要，可以简单地删除。这里，离群值的重要性通过最大离群值与量化尺度（等式 1 中的 s ）之间的比率来衡量。比率越大表示激活分布越分散，从而导致量化误差越显著。llm.npu 在离线阶段使用大型语料库数据来描述这些异常值的重要性（图 12），并修剪大部分不重要层的异常值。通常，我们观察到输入和输出附近的层具有更高的重要性。这是因为靠近输入的层很容易受到令牌差异的影响，表现出更大的波动，而靠近输出的层很容易积累来自浅层的误差。根据观察，llm.npu 通过离线分析修剪了前 85% 最不重要的异常值，从而消除了 CPU-NPU 同步。

3.4 子图乱序执行

正如 (§2.3) 中所述，LLM 量化算法无法完全消除浮点运算，因此 llm.npu 将其执行流程划分为 NPU 和 CPU/GPU 协作。通常，LayerNorm、Attention 以及阴影异常值计算都放在 CPU/GPU 上；

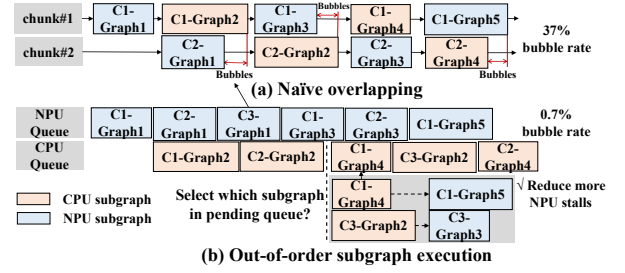


图 13. 无序子图执行。

而其他线性层则在 NPU 上处理。然而，我们发现简单地重叠它们的执行效率低下，导致较大的执行泡沫（关键路径中的泡沫率为 37%），如图 13 (a) 所示。

无序执行。为了减少这些执行气泡，llm.npu 以一个关键见解为指导，即在块和子图级别上进行分区后，LLM 子图可以以无序方式进行调度。更具体地说，任何输入就绪的子图都可以在不严格遵循块序列的情况下执行。例如，当 C2-Graph1 完成时，可以在冒泡期间执行第三个块（C3-Graph1）的第一个子图。

为了保持正确性，llm.npu 考虑两种类型的依赖关系：

- (1) 跨块依赖关系。像注意力这样的运算符依赖于先前块的数据。这意味着第 i 个块 j 个子图 $G_{i,j}$ 取决于 $0, 1, \dots, i-1$ 个块的第 $j-1$ 个子图：

$$G_{i,j} \leftarrow G_{0,j-1}, G_{1,j-1}, \dots, G_{i-1,j-1} \quad (2)$$

- (2) 块内依赖。LayerNorm、Linear 和 Quantize 等运算符仅依赖于同一块内的先前子图。因此，第 i 个块的第 j 个子图 $G_{i,j}$ 取决于同一块的第 $j-1$ 个子图：

$$G_{i,j} \leftarrow G_{i,j-1} \quad (3)$$

由于移动处理器在并行性和抢占方面较弱 [40, 86, 97]，为了确保效率，处理器在任何给定时间只能执行一个子图：

$$\sum_{i=0}^N \sum_{j=0}^M P_{i,j,t} = 1, \forall t \quad (4)$$

其中 $P_{i,j,t} = 1$ 表示子图 $G_{i,j}$ 在时间 t 在处理器 P 上运行，而 N 和 M 分别表示块和子图的最大数量。llm.npu 的目标是找到一个执行顺序，在这些约束下最小化所有子图的总执行时间。不幸的是，这个调度问题可以简化为经典的 NP 困难旅行商问题 [44]。而且，由于块的数量随用户提示而变化，无法离线生成最优调度策略。

相反，llm.npu 使用在线启发式算法。关键思想是不要关注执行时间

子图 g ，但关于执行 g 如何有助于减少 NPU 停顿，其动机是观察到在预填充阶段，NPU 执行时间通常主导推理延迟，是关键路径。例如，使用 Qwen1.5-1.8B 模型，提示长度为 256，NPU 执行需要 315ms，大约是 CPU 的两倍。

具体来说，我们定义子图 g 对减少 NPU 停顿的贡献如下：如果子图 g 要在 CPU/GPU 上执行，则令 S 为在 g 完成后可以执行的新子图的集合。 S 将在 NPU 上执行。 S 较长的执行时间有利于减少 NPU 停顿。因此， g 的贡献被定义为 S 的总执行时间。相反，如果 g 在 NPU 上执行，则 S 的执行时间较短是有利的， S 的执行时间的负值作为 g 的贡献，公式为：

$$C = \begin{cases} \sum T_i, \forall i \in S \text{ if } g \text{ is on the CPU/GPU} \\ -\sum T_i, \forall i \in S \text{ if } g \text{ is on the NPU} \end{cases} \quad (5)$$

其中 T 是子图执行时间。llm.npu 始终选择具有最大 C 的子图，这意味着具有 S 的子图 g 在 NPU 上的执行时间最长，或者在 CPU/GPU 上的执行时间最短。

简而言之，llm.npu 在准备阶段离线分析所有子图执行时间及其依赖关系。在预填充阶段，它会计算所有待处理的子图 C 值，并选择具有最大 C 的子图来运行，性能开销为微秒级。

4 实施与评估

我们已经为 Qualcomm Hexagon NPU 全面实现了 llm.npu，其中包含 10K 行 C/C++ 和汇编语言代码。我们选择高通 SoC 作为目标平台，因为它在移动设备上的受欢迎程度和强大的 NPU 能力。Qualcomm Hexagon 也是唯一一款具有开放指令集架构的移动 NPU。llm.npu 构建于 MLLM [23]（一种最先进的移动 LLM 引擎）和 QNN 框架 [25]（高通神经处理 SDK）之上。它支持从 Hugging Face [21] 导出的标准 LLM 格式。为了促进 LLM 执行，除了 QNN 支持的操作之外，我们还实现了 KVCache、SiLU、RMSNorm、ROPE 等特定操作。为了减少 CPU/GPU 和 NPU 之间的上下文切换开销，llm.npu 利用共享缓冲区来同步来自不同处理器的中间结果。对于端到端推理，llm.npu 与任何解码引擎兼容，并利用 MLLM CPU 后端进行解码阶段，易于实现，默认块长度为 256。异常层的默认剪枝率为 85%。llm.npu 在当前实现中不考虑资源检测和争用。

该原型还包含两项优化。(1) 我们广泛的实验表明，移动 NPU 通常更喜欢 CNN 架构中的张量大小（例如，相等的“高度”和“宽度”）。例如，权重为的线性层

2048×2048 对于 1024×1×2048 和 32×32×2048 的输入产生相同的结果，但使用 32×32×2048 可以将执行延迟减少 1.62×。因此，llm.npu 在准备阶段会分析线性层所有可能的等效形状，并选择最有效的形状。(2) 移动 NPU 通常访问有限的内存区域（例如，Hexagon NPU 为 4GB），该内存区域可能小于 LLM 权重的大小。为了在有限的内存内最大化预填充加速，llm.npu 优先在 NPU 上执行计算密集型任务，例如 FFN，以提高效率。

4.1 实验设置

硬件设置。我们在两部配备不同 Qualcomm SoC 的智能手机上测试 llm.npu：Redmi K70 Pro（Snapdragon 8gen3，24GB 内存）和 Redmi K60 Pro（Snapdragon 8gen2，16GB 内存）。所有设备均运行 Android OS 13。

模型和数据集。我们使用各种典型的移动大小的 LLM 进行了测试：Qwen1.5-1.8B [27]、Gemma-2B [9]、Phi2-2.7B [16]、LLaMA2-Chat-7B [11] 和 Mistral-7B [14]。为了评估 llm.npu 的量化精度，我们采用了广泛认可的 LLM 基准，包括 LAMBADA [68]、HellaSwag [103]、Wino Grande [4]、OpenBookQA [62] 和 MMLU [43]。对于推理速度实验，我们选择了来自 Longbench、2wikimqa 和 TriviaQA [28] 的基于检索的数据集，用于模拟上下文感知生成任务，例如自动电子邮件回复。此外，我们使用 DroidTask 数据集 [22] 评估了 llm.npu 的屏幕问答和将指令映射到 UI 操作场景，以模拟基于代理的 UI 自动化任务。

基线。我们主要将 llm.npu 与 5 个基线进行比较，包括 3 个广泛使用的移动 LLM 引擎（TFLite [57]、MNN [49] 和 llama.cpp [59]）。这些引擎仅支持移动 CPU 和 GPU；2 个高级基线是 MLC-LLM [75]（一种用于设备上 GPU 的 LLM 编译器）和 PowerInfer-v2（也利用移动 NPU 来加速预填充 [94]）。由于 PowerInfer-v2 不是开源的，我们使用其论文中报告的数据。需要注意的是，这些基线通常仅支持我们评估的 5 个**大语言模型**。

指标。我们主要测量 LLM 推理精度、预填充延迟、预填充能耗、预填充内存消耗和端到端推理延迟。能耗是通过 Android 操作系统中的 /sys/class/power_supply 每 100ms 进行一次分析来获得的。所有实验重复三次，我们报告平均数。

4.2 预填充性能。

我们在两台设备上以 64、256 和 1024 个令牌的提示长度评估 llm.npu 的预填充性能（速度和能耗），如图 14 和 15 所示。尽管数据集之间存在异常变化，但对预填充性能的总体影响很小，因此我们报告来自 Long-Bench 数据集的结果。结果表明，llm.npu 在这两个指标上始终优于所有基线，并且随着提示长度的增加，优势变得更加明显。

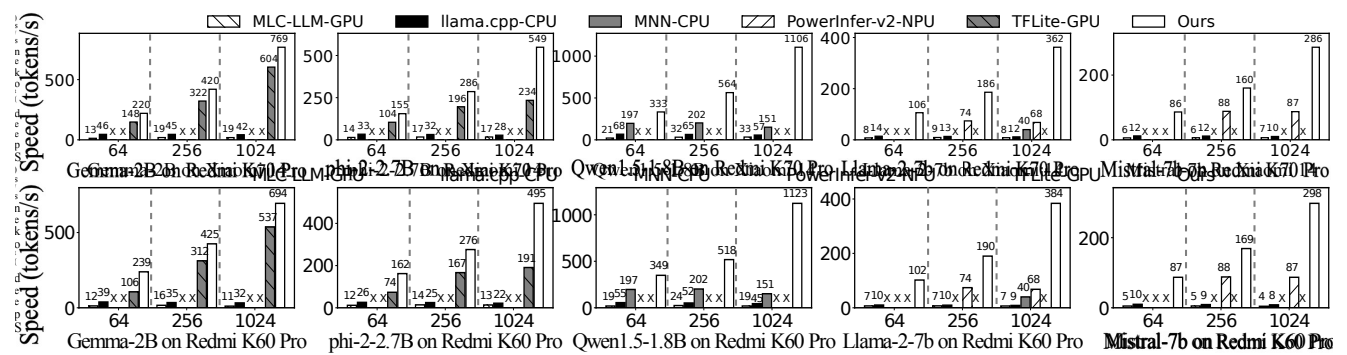


图 14. 不同设备上不同提示长度下的预填充速度（数据集：Longbench-2wiki-Multi-doc QA）。

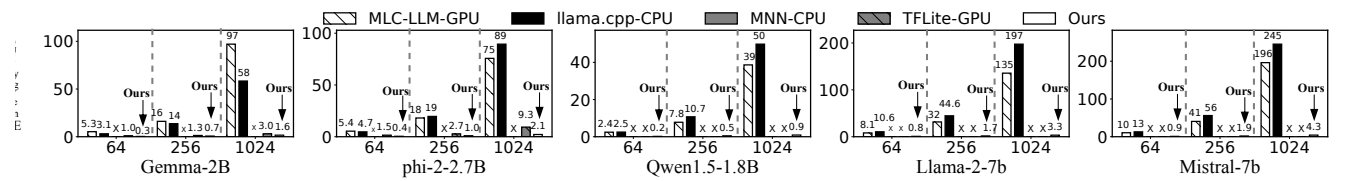


图15 Redmi K60 Pro不同提示时长下的能耗（数据集：Longbench-2wiki-Multi-doc Q 一个）。

预充速度。对于 1024 个令牌的提示长度，与 llama.cpp-CPU、MNN-CPU、MLC-GPU、TFLite-GPU 相比，llm.npu 在 Redmi K70 Pro 上可分别将预填充延迟减少 18.17–38.4×、7.3×、32.5–43.6× 和 1.27–2.34×。在 Redmi K60 Pro 上，这些改进分别为 21.3–41.3×、7.43×、37.2–69.3× 和 1.3–2.6×。这些加速归功于 llm.npu 使用了三种充分利用移动 NPU 的创新技术，包括阴影离群值执行、高效每张量 MatMul 和乱序子图执行。与同样使用 NPU 进行预填充的基线 PowerInfer-V2-NPU 相比，llm.npu 通过采用 NPU 友好的 INT8 线性计算和细粒度子图调度 (§3.4)，可以在两个设备上分别实现 3.28–5.32× 和 3.4–5.6× 加速。

对于 64 个 token 的提示长度，llm.npu 的预填充速度比 llama.cpp-CPU、MNN-CPU、MLC-GPU 快 14.86–7.10×、1.69×、10.91–17.32×、1.48× 和 1.81–2.51×，TFLite-GPU 和 PowerInfer-V2-NPU 的平均加速比 1024 个令牌提示低 10.5×、4.31×、2.68×、1.02× 和 1.96×。这是因为较短的提示可能会导致填充问题并限制 llm.npu 的乱序执行调度效率。

预填充能量消耗。能耗评估是在唯一可 Root 的设备 Redmi K60 Pro 上进行的。由于缺乏能耗数据和开源代码，PowerInfer-V2 被排除在外。对于 1024 个令牌提示，与 llama.cpp-CPU、MLC-GPU 和 TFLite-GPU 相比，llm.npu 分别将能耗降低了 35.63–59.52×、35.21–59.25× 和 1.85–4.32×。对于 64 个令牌提示，节省的费用为 10.38–14.12×、10.38–17.79× 和 3.22–3.67×，

分别。这些节省归功于移动 NPU 的高能效以及 llm.npu 的三项新技术，可最大限度地提高 NPU 性能。通常，在 LLM 预填充阶段，所有 CPU 核心都被充分利用，消耗的功率最高；NPU 的运行频率为 500-750 MHz，功耗最低。

4.3 端到端性能

我们使用三种工作负载根据基准系统评估 llm.npu 的实际性能：DroidTask 数据集上的 UI 自动化、LongBench 数据集上的上下文感知自动电子邮件回复、Persona-Chat 数据集上的聊天摘要。表 5 显示了三个数据集的提示和输出长度，以及相应的预填充、解码和端到端推理延迟结果。表 5 中的端到端加速是每个样本的几何平均值。我们的主要观察结果是 llm.npu 始终在所有三个数据集上实现最低的推理延迟。

对于 LongBench 数据集，llm.npu 显示出显著的速度改进：比 llama.cpp-CPU 提高 23.0–46.2×，比 MLC-LLM-GPU 提高 16.5–36.4×，比 MNN-CPU 提高 4.08–4.19×，比 PowerInfer-V2-NPU 提高 3.51–3.73×，以及 1.27–2.03× 通过 TFLite-GPU。这种令人印象深刻的表现主要归功于 llm.npu 在预填充阶段的卓越效率。相对于 TFLite-GPU 的加速比较低，因为 llm.npu 目前依赖 CPU 后端进行解码，没有优化，而 TFLite 使用 GPU。值得注意的是，llm.npu 与任何解码引擎兼容，这意味着一旦 TFLite 开源，llm.npu 就可以将其集成为解码后端，从而有可能进一步提高性能。

表 5. 在 Redmi K70 Pro 上使用真实移动应用程序执行进行的端到端延迟比较。每个单元按以下格式组织：端到端延迟（预填充延迟、解码延迟）。单位：秒。

LLM	Datasets	MLC	LCPP	MNN	PI	TFLite	Ours	Speedup
Qwen1.5-1.8B	Longbench: 2wiki-Multi-doc QA prompt length: 1451–1672 tokens output length: 2–4 tokens	45.6 (45.43, 0.17)	26.7 (26.43, 0.24)	10.6 (10.01, 0.59)	-	-	1.7 (1.49, 0.24)	6.2-26.8×
Gemma-2B		78.4 (78.03, 0.32)	34.6 (34.31, 0.25)	-	-	2.6 (2.40, 0.19)	1.9 (1.68, 0.25)	1.4-41.3×
Phi-2-2.7B		87 (86.70, 0.28)	53.3 (53.00, 0.34)	13 (11.98, 1.05)	-	6.3 (6.06, 0.28)	3.1 (2.79, 0.34)	2.0-28.1×
LlaMA-2-7B		184.7 (184.20, 0.45)	146 (145.25, 0.70)	22.4 (22.35, 0.05)	19.8 (19.04, 0.78)	-	5.3 (4.62, 0.70)	3.7-34.8×
Mistral-7b		254.2 (253.45, 0.72)	200.2 (199.37, 0.87)	-	20 (19.22, 0.78)	-	5.5 (4.61, 0.87)	3.6-46.2×
Geo-mean (speedup)		34.7×	21.8×	4.8×	3.7×	1.7×	-	
Qwen1.5-1.8B	Longbench: TriviaQA prompt length: 1511–1787 tokens output length: 5–11 tokens)	46 (45.68)	27 (26.57, 0.47)	11.2 (10.07, 1.15)	-	-	2.0 (1.49, 0.47)	5.6-23.0×
Gemma-2B		81.8 (81.25, 0.26)	36.2 (35.72, 0.43)	-	-	2.8 (2.50, 0.32)	2.2 (1.75, 0.43)	1.3-37.2×
Phi-2-2.7B		91.4 (90.94, 0.50)	56.3 (55.60, 0.68)	14.7 (12.57, 2.11)	-	6.8 (6.36, 0.40)	3.6 (2.94, 0.68)	1.9-25.4×
LlaMA-2-7B		197.3 (196.43, 0.83)	156.2 (154.90, 1.28)	23.8 (20.83, 3.00)	21.8 (20.31, 1.34)	-	6.2 (4.93, 1.28)	3.5-31.8×
Mistral-7b		266.2 (264.93, 1.31)	210 (208.41, 1.57)	-	21.5 (20.10, 0.41)	-	6.4 (4.82, 1.57)	3.4-41.6×
Geo-mean (speedup)		31.0×	19.6×	4.4×	3.4×	1.6×	-	
Qwen1.5-1.8B	DroidTask: clock prompt length: 656–827 tokens output length: 1–5 tokens)	21 (20.84, 0.14)	10.4 (10.16, 0.25)	3.9 (3.25, 0.61)	-	-	1.4 (1.16, 0.25)	2.80-15×
Gemma-2B		39.4 (39.11, 0.25)	16.5 (16.25, 0.21)	-	-	2.5 (2.30, 0.18)	1.2 (0.97, 0.23)	2.1-32.8×
Phi-2-2.7B		46.6 (46.40, 0.24)	25 (24.62, 0.34)	7.4 (6.34, 1.06)	-	4.2 (3.98, 0.2)	3.1 (2.72, 0.34)	1.4-15.0×
LlaMA-2-7B		87.7 (87.38, 0.33)	60.4 (59.83, 0.63)	10.6 (8.58, 2.11)	11.1 (10.74, 0.37)	-	4.8 (4.28, 0.53)	2.2-18.3×
Mistral-7b		122.3 (121.82, 0.51)	68.6 (67.91, 0.65)	-	12 (11.64, 0.37)	-	4.9 (4.26, 0.65)	2.4-25.0×
Geo-mean (speedup)		20.2×	10.8×	2.4×	2.4×	1.7×	-	
Qwen1.5-1.8B	DroidTask: clock prompt length: 505–645 tokens output length: 3–5 tokens	16.2 (16.05, 0.14)	8.1 (7.83, 0.25)	3.1 (2.5, 0.61)	-	-	1.1 (0.90, 0.25)	2.8-14.7×
Gemma-2B		29.4 (29.14, 0.52)	12.3 (12.11, 0.20)	-	-	1.9 (1.71, 0.18)	0.9 (0.72, 0.23)	2.1-32.7×
Phi-2-2.7B		35.4 (35.15, 0.23)	19 (18.65, 0.34)	5.9 (4.80, 1.05)	-	3.2 (3.01, 1.2)	2.4 (2.07, 0.34)	1.3-14.8×
LlaMA-2-7B		63.7 (63.38, 0.33)	43.9 (43.40, 0.52)	7.7 (5.06, 1.64)	8.2 (7.79, 0.37)	-	3.6 (3.10, 0.52)	2.1-17.7×
Mistral-7b		90.1 (89.60, 0.51)	50.6 (49.95, 0.65)	-	8.9 (8.56, 0.37)	-	3.8 (3.13, 0.65)	2.3-23.7×
Geo-mean (speedup)		19.7×	10.5×	2.5×	2.3×	1.7×	-	
Qwen1.5-1.8B	Persona-Chat prompt length: 488–584 tokens output length: 35–57 tokens	18.74 (16.54, 2.20)	11.86 (8.07, 3.79)	11.87 (2.58, 9.29)	-	-	6.72 (2.92, 3.80)	1.8-2.8×
Gemma-2B		30.9 (27.05, 3.82)	14.3 (11.24, 3.02)	-	-	4.3 (1.59, 2.67)	4.2 (0.67, 3.48)	1.02-7.4×
Phi-2-2.7B		36.1 (32.51, 3.57)	22.5 (15.24, 3.22)	20.6 (4.43, 16.19)	-	5.9 (2.79, 3.06)	5.1 (1.91, 3.22)	1.2-7.1×
LlaMA-2-7B		64.2 (58.96, 5.20)	48.7 (40.37, 8.30)	45.5 (7.15, 38.23)	13.0 (7.25, 5.77)	-	11.2 (2.88, 8.30)	1.2-5.7×
Mistral-7b		90.5 (82.54, 8.0)	56.3 (46.02, 10.28)	-	13.7 (7.89, 5.76)	-	13.2 (2.88, 10.28)	1.04-6.9×
Geo-mean (speedup)		10.5×	3.5×	3.1×	1.1×	1.1×	-	

*第一行中的LCPP和PI分别代表llama.cpp和PowerInfer-V2。

对于 DroidTask 数据集，与 llama.cpp-CPU 相比，llm.npu 将端到端推理延迟减少了 7.9–12.5%，

与 MLC-LLM-GPU 相比为 15.0–32.8%，与 MNN-CPU 相比为 2.38–2.45%，与 PowerInfer-V2-NPU 相比为 2.27–2.44%，与 TFLite-GPU 相比为 1.35–2.38%。由于 UI 自动化中的提示与电子邮件编写相比较短，DroidTask 数据集的性能提升略小。

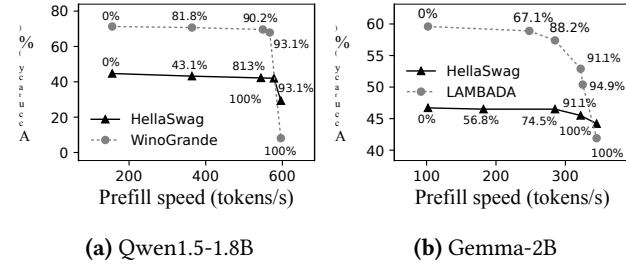


图 16. 不同异常值修剪率的生成速度和准确性。位于图上方或下方的文本表示相应的修剪率。

对于 Persona-Chat 数据集，与 llama.cpp-CPU 相比，llm.npu 将端到端推理延迟平均降低了 3.5 $\times$ ，与 MLC-LLM-GPU 相比，平均降低了 10.5 $\times$ ，与 MNN-CPU 相比，降低了 3.1 $\times$ ，与 PowerInfer-V2-NPU 相比，降低了 1.1 倍，与 TFLite-GPU 相比，降低了 1.1 $\times$ 。相对于其他数据集，性能改进下降的主要原因是该数据集生成的输出标记数量较多。由于 llm.npu 目前仅支持基于 CPU 的解码，因此较慢的解码后端会导致端到端延迟增加。这个问题可以通过使用移动 GPU 作为解码后端来缓解（详细信息请参见第 4.6 节）。

4.4 推理精度

总体准确度。我们研究了 llm.npu 在 5 个 LLM 基准上的推理准确性：LAMBADA [68]、MMLU [43]、WinoGrande [4]、OpenBookQA [62] 和 HellaSwag [103]。为了进行比较，我们评估了 4 个替代方案：FP16（非量化）、K-Quant [59]（在 llama.cpp 中使用）、SmoothQuant（最先进的每张量方法）[84] 和 LLM.Int8()（最先进的浮点异常值处理方法）[33]。llm.npu 的精度损失可以忽略不计，并且显著优于其他量化算法，如表 6 所示。

具体来说，llm.npu 的平均准确度仅比 FP16 低 1%，但与 SmoothQuant 相比，准确度提高了 32.9%，比 K-Quant 提高了 70.9%。SmoothQuant 使用静态分析将离群值平滑至正常值，这一改进归功于 llm.npu 以 CPU 浮点精度动态处理离群值位置。llm.npu 在元素级别解决异常值，提供比使用组级别量化尺度的 K-Quant 更高的精度。此外，llm.npu 实现了与 LLM.Int8() 相当的精度（平均损失 0.1%），因为两者都以浮点精度处理异常值。但 llm.npu 更好地利用了 NPU 特定的计算特性，保持了高精度和 NPU 效率。

准确性和性能之间的权衡。llm.npu 通过调整离群剪枝率来平衡生成精度和速度，较高的剪枝率会导致

表 6. llm.npu 和基准上的 LLM 能力准确性。“SQ”：SmoothQuant；“Int8()”：LLM.Int8()；“Degrad.”：与 FP16 相比，精度下降。

LAMBADA	FP16	SQ	INT8()	K-Quant	Ours	Ours Degrad.
Qwen1.5-1.8B	71.1%	65.6%	71.0%	62.7%	71.7%	+0.6%
Gemma2-2B	59.6%	45.8%	59.2%	56.9%	59.4%	-0.2%
Phi-2-2.7B	72.2%	66.1%	71.7%	59.3%	67.5%	-4.7%
LlaMA-2-7B	87.5%	71.9%	88.0%	15.6%	86.3%	-1.2%
Mistral-7b	84.8%	51.2%	85.3%	23.9%	84.1%	-0.7%
Avg. Degrad.	-	-14.9%	0%	-31.3%	-1.2%	

HellaSwag	FP16	SQ	INT8()	K-Quant	Ours	Ours Degrad.
Qwen1.5-1.8B	43.8%	40.9%	43.5%	44.3%	43.8%	0%
Gemma2-2B	46.5%	43.8%	46.1%	45.4%	47.3%	+0.8%
Phi-2-2.7B	48.2%	46.2%	47.7%	47.6%	46.9%	-1.3%
LlaMA-2-7B	52.8%	44.4%	53.1%	50.5%	53.5%	+0.7%
Mistral-7b	57.4%	44.9%	57.9%	57.0%	57.0%	-0.4%
Avg. Degrad.	-	-5.7%	-0.1%	-0.8%	-0.0%	

WinoGrande	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	58.3%	51.0%	58.2%	59.0%	59.3%	+1.0%
Gemma2-2B	58.3%	54.8%	59.0%	58.5%	59.5%	+1.2%
Phi-2-2.7B	72.2%	68.9%	72.4%	72.5%	70.2%	-2.0%
LlaMA-2-7B	65.2%	56.9%	66.2%	67.4%	65.1%	-0.1%
Mistral-7b	73.5%	59.1%	73.3%	73.5%	73.1%	-0.4%
Avg. Degrad.	-	-7.4%	+0.3%	+0.7%	-0.1%	

OpenBookQA	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	28.8%	23.0%	28.5%	28.0%	26.6%	-2.2%
Gemma2-2B	33.7%	28.0%	34.2%	33.0%	38.4%	+4.7%
Phi-2-2.7B	41.0%	35.9%	40.2%	39.5%	37.7%	-3.3%
LlaMA-2-7B	32.7%	25.0%	32.0%	31.5%	31.1%	-1.6%
Mistral-7b	39.4%	25.6%	39.3%	37.9%	39.3%	-0.1%
Avg. Degrad.	-	-7.6%	-0.3%	-1.1%	-0.5%	

MMLU	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	29.7%	27.9%	29.1%	29.8%	30.8%	+1.1%
Gemma2-2B	35.7%	32.1%	35.1%	35.1%	36.4%	+0.7%
Phi-2-2.7B	35.4%	35.3%	35.6%	35.7%	36.7%	+1.3%
LlaMA-2-7B	37.8%	29.2%	38.1%	34.4%	36.9%	-0.9%
Mistral-7b	42.1%	30.9%	41.4%	42.3%	41.0%	-1.1%
Avg. Degrad.	-	-5.1%	-0.3%	-0.7%	+0.2%	

生成速度更快，但质量较低。为了全面展示离群剪枝率如何影响生成精度和速度，我们在 Redmi K70 Pro 上使用 Qwen1.5-1.8B 和 Gemma-2B 以及 HellaSwag [103] 和 LAMBADA [68] 数据集进行了实验。结果如图 16 所示。

当没有异常值被剪枝时（剪枝率=0%），llm.npu 实现了最高的准确率，尽管生成速度最慢。具体来说，Qwen1.5-1.8B 模型在 Lambada 数据集上可以达到 71.3% 的准确率，相应的生成速度为每秒 156 个令牌。相反，Gemma-2B 模型的准确度为 59.6%，生成速度为每秒 102 个令牌。在修剪 80% 的异常值后，Qwen1.5-1.8B 模型的准确率下降至 42.2%，而其生成速度则增加至每秒 54 4 个令牌。同样，Gemma-2B 模型的准确率下降至 57.4%，但其生成速度却上升至 285

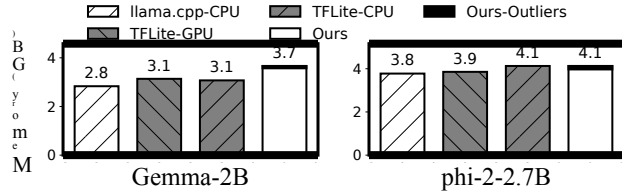


图 17. 不同基线的内存消耗（提示长度=512）。

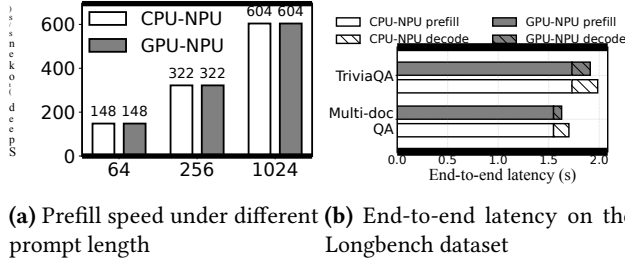


图 18. CPU-NPU 协调和 GPU-NPU 协调之间的预填充速度和端到端延迟比较。

每秒令牌数。当所有异常值都被修剪时，生成精度达到最低点，而生成速度达到最高点。具体来说，Qwen1.5-1.8B模型的准确率仅为8.1%，生成速度为596个token/s；Gemma-2B模型的准确度为41.9%，生成速度为345个令牌/秒。

4.5 内存消耗

我们将 llm.npu 与 INT8 权重基线进行比较，因为移动 NPU 仅支持 INT8 权重计算。图 17 显示了使用 512 令牌提示的 Redmi K60 Pro 上的内存消耗结果。llm.npu 比 llama.cpp 和 TFLite 多消耗 $1.32 \times$ 的内存。开销是由于 ML LM 和 QNN 框架造成的，它们为每个运算符分配独立的激活缓冲区以提高速度。llm.npu 引入的微小额外内存开销是其 §3.3 阴影离群值执行技术（黑色），该技术将微小的浮点权重加载到内存中，仅占总内存的 0.6%–1%。

4.6 GPU-NPU协调分析

llm.npu 与移动 GPU 兼容；不过，由于实现简单，我们目前只实现了 CPU-NPU 版本。为了说明使用移动 GPU 的潜在优势，我们模拟 GPU-NPU 协同调度，假设 TFLite 作为 GPU 后端。使用 Gemma-2B 模型在不同的提示长度和 Longbench 数据集上进行评估，特别是在 Redmi K70 Pro 上，如图 18 所示。我们的观察表明，虽然 GPU-NPU 协调

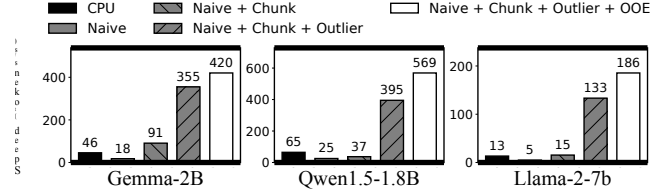


图 19. llm.npu 的消融研究（提示长度=512）。

不提升预填充速度，有效降低端到端时延。

如图18(a)所示，通过GPU-NPU协调实现的预填充速度与CPU-NPU协调的预填充速度相当。出现这种现象的原因是，在 llm.npu 的设计中，对移动 CPU 和 GPU 的计算需求明显低于对 NPU 的计算需求（详见 3.4 节）。因此，移动CPU和GPU的计算延迟可以被NPU计算有效隐藏，这表明移动CPU或移动GPU的选择并不重要。另一方面，与CPU-NPU协调相比，GPU-NPU协调可以将端到端延迟减少80-90毫秒。这一改进归功于 GPU 相对于 CPU 更快的解码速度。

4.7 消融研究。

我们对使用 Qwen1.5-1.8B、Gemma-2B 和 LLaMA2-7B 模型的 llm.npu 每种技术带来的好处进行了全面的细分分析，如图 19 所示。最左边的条代表使用 llama.cpp 在 CPU 上的预填充速度。第二个条显示了移动 NPU 上的简单实现，然后是我们三种技术的结合，最右边的条代表 llm.npu。这三种技术分别用 chunk (§3.2)、outlier (§3.3) 和 OOE (§3.4) 表示。我们观察到 llm.npu 的三种技术对整体改进做出了重大贡献。

首先，由于 LLM 和移动 NPU 之间存在巨大差距，直接将 LLM 预填充工作负载卸载到移动 NPU 会导致 2.55 – $2.68 \times$ 延迟，如 §2.3 中详述。此外，块共享图通过减少图构建和优化延迟，将预填充速度提高了 1.46 – $5.09 \times$ 。Gemma-2B 模型实现了最高的加速，因为它需要更多的时间来构建和优化。此外，阴影异常值执行将预填充延迟减少了 3.91 – $8.68 \times$ ，允许每个张量 MatMul 操作以最小的 CPU/GPU 开销充分利用 NPU。最后，无序子图执行通过减少 NPU 内的执行气泡，将预填充延迟减少了 18% – 44% 。

5 讨论和未来的工作

NPU 相对于 GPU 的优势。通常，移动 GPU 和 NPU 都充当设备上的硬件加速器

DNN 推理 [53, 86]。然而，由于移动 GPU 已经广泛用于设备上的图形渲染，依赖它们进行 LLM 推理可能会导致严重的资源争用。相比之下，专门用于深度学习任务的移动 NPU 的工作负载相对较轻，因此有充足的时间来运行 LLM 推理。此外，如第 2.2 节所述，与移动 GPU 相比，移动 NPU 提供了更大的计算能力、更高的能源效率和更高的可用性，使它们更适合设备上的 LLM 推理。

llm.npu 的技术稳定性。尽管最近的趋势是在高级 NPU 中支持 FP16 [6,13,17]，但 llm.npu 对于基于 NPU 的预填充加速仍然至关重要，原因有两个：（1）整数运算由于其卓越的计算性能、内存效率和节能而有望在移动 NPU 端 LLM 推理中保持主导地位，而 CPU/GPU 作为补充单元来提高准确性。因此，llm.npu 的三种 INT8 相关技术，包括跨 CPU/GPU 和 NPU 的协同调度以实现高精度和高效率，将继续对该领域产生重大影响。（2）llm.npu 能够为传统设备提供设备上的 LLM 功能，从而实现 LLM 访问的民主化。

未来硬件设计的影响。llm.npu 的性能可以通过以下硬件优化得到进一步增强：（1）动态形状感知优化：能够动态重新配置以适应不同的输入和输出形状的硬件将显著提高 LLM 推理的加速。（2）增加数据缓存大小：移动 NPU 通常通过将整个层的权重加载到数据缓存中来增强推理性能。增加 NPU 缓存大小以与 LLM 权重大小保持一致对于获得最佳性能至关重要。（3）计算单元中的混合精度操作数：移动 NPU 通常仅支持统一精度操作数，需要额外的数据格式转换来执行 FP16 * INT8 运算。通过在计算单元中启用对混合精度操作数的支持可以消除这种开销。

带有移动 GPU 的 llm.npu。由于易于实现，我们当前的实现仅依赖于移动 CPU 进行浮点运算符和解码阶段。不过，llm.npu 的设计同时兼容移动 CPU 和 GPU。我们计划在未来的开发中纳入 GPU-NPU 协调。将移动 GPU 集成到 llm.npu 的主要工程工作包括配置 GPU 后端和实现必要的运算符。借助移动 GPU，llm.npu 可以显著减少端到端推理延迟，如第 4.6 节所述。

6 相关工作

设备上 LLM 优化。为了减少设备上 LLM 推理的大量内存消耗，各种

已经提出了压缩技术，包括量化和知识蒸馏[30,35,38,46,58,65,70,80,88,91,92,96,98,101]。为了最大限度地减少设备上的 LLM 计算，研究人员引入了令牌修剪 [29,31,51,72,79]，它在推理过程中修剪不必要的令牌。推测解码是一种通过将任务卸载到较小的 LLM 来加速令牌生成的方法，已在开源框架中广泛采用[3, 59]，并进行了广泛的研究[32, 36, 42, 50, 61, 87, 95]。作为系统优化，llm.npu 是正交的并且兼容这些算法级别的优化。

ML 的片上卸载。人们已经对此进行了深入研究，以便通过利用 GPU 和 NPU 等异构移动处理器来实现更快的 DNN 推理 [37, 39, 41, 45, 52–54, 63, 66, 85, 86, 89, 93, 94, 104, 105]。MobiSR [54]利用移动 NPU 来加速超分辨率计算。然而，这些方法并没有解决 LLM 特定的功能，并且不适合设备上的 LLM 场景。最相关的工作是 PowerInfer-V2 [94]，它也利用移动 NPU 进行预填充。但是，它没有全面分析 LLM 预填充的设备上 NPU 卸载的挑战，也没有结合技术来充分利用 NPU 功能。我们的实验表明，我们的方法在预填充阶段的效率提高了 5.32×。llm.npu 受到这些努力的启发，是第一个具有高效设备上 NPU 卸载的 LLM 推理框架。移动 NPU 执行优化。随着移动 NPU 在智能手机中的重要性日益增加，人们付出了巨大的努力来优化其执行效率 [34,64,71,73,76,78]。Pitchfork [73]定义了一种可移植的定点中间表示来优化定点执行效率。Isaria [76]提出了一个框架，用于自动生成 DSP 架构的矢量化编译器，为移动 NPU 创建高效的运算符代码。作为一个系统框架，llm.npu 是正交的，可以利用它们生成更高效的算子库作为执行后端，进一步提高性能。

7 结论

本文提出了 llm.npu，这是第一个利用设备上 NPU 卸载来减少预填充延迟和能耗的 LLM 推理系统。llm.npu 融入了新技术：块共享图、影子离群值执行和乱序子图执行，以提高 NPU 卸载效率。大量实验证明 llm.npu 具有卓越的性能优势，例如高达 43.6× 的加速和 59.5× 的节能。

8 致谢

该工作得到了国家自然科学基金委（No. 62325201）和华为大学联合研究项目的资助。

神器附录

A.1 摘要

人工评估主要呈现第 4.2 节中详述的预填充性能结果和第 4.4 节中讨论的推理准确性评估结果。这些结果可以在我们的 GitHub 存储库¹ 的 Performance_results 和 Accuracy_results 目录中找到。要进行这些评估，请参阅存储库中的 README.md 文件。存储库中的每个评估目录都包含有关安装和执行 llm.npu 和所有基线方法评估的详细说明。这包括环境设置、LLM 和数据集下载、源代码编译和执行以及结果清晰度的指导。我们建议首先进行精度评估，然后进行预填充性能测试。为了获得最佳结果，请使用 Redmi K70 Pro 24G 进行设备上 LLM 推理，并使用 NVIDIA A100 40G GPU 服务器进行测试，因为这些设置已经过作者的验证。大多数实验都以 Qwen1.5-1.8B 为代表示例来演示 llm.npu 的功能和评估结果。如果需要，用户可以通过调整代码灵活地定制自己的法学硕士模型。一旦工件评估被接受，我们将存档评估结果。

A.2 工件检查表（元信息）

- 编译：支持用户编译。
- 二进制：已编译的二进制文件可用。
- 示范模型采用型号：Qwen1.5-1.8B-Chat。还支持 Gemma-2B、Phi-2-2.7B、LLaMA-2-7B 和 Mistral-7B。
- 数据集：HellaSwag [103] 和 LAMBADA [68] 用作演示数据集。还支持 OpenBookQA [62]、WinoGrande [4] 和 MMLU [43]。
- 服务器硬件：NVIDIA-A100 40G
- 设备硬件：Redmi K70 Pro 24G
- 执行：C++、Python 和 bash 脚本
- 指标：推理准确性和预填充速度
- 输出：登录终端
- 实验：第 4.2 节和第 4.4 节
- 需要多少磁盘空间（大约）：200GB
- 准备工作流程需要多长时间（大约）？：5 小时
- 完成实验需要多长时间（大约）：2 小时
- 是否公开？：是
- 代码许可证（如果公开可用）？：麻省理工学院
- 数据许可证（如果公开可用）？：麻省理工学院
- 使用的工作流自动化框架？：mllm [23]
- 已存档（提供 DOI）？：https://doi.org/10.5281/zenodo.14392760。

A.3 描述

A.3.1 如何访问。我们的人工评估可公开访问

¹https://doi.org/10.5281/zenodo.14392760

https://doi.org/10.5281/zenodo.14392760。

A.3.2 硬件依赖性。对于第 4.4 节中讨论的准确性结果，审核者将需要访问配备 NVIDIA A100 40GB GPU、x86 CPU 内核以及最好超过 200GB CPU 内存的 GPU 服务器。服务器应在 Ubuntu 20.04 操作系统上运行。

为了评估第 4.2 节中的性能结果，评测人员需要配备 24GB 内存的 Redmi K70 Pro。

A.3.3 软件依赖性。为了获得第 4.4 节中概述的准确性结果，服务器应安装 CUDA 12.5 和具有 Python 3.8 或更高版本的 Conda 虚拟环境。

A.3.4 数据集。用于准确性结果的数据集包括 HellaSwag [103]、LAMBADA、OpenBookQA [62]、WinoGrande [4] 和 MMLU [43]。所有数据集都可以使用我们的 Hugging Face [21] 代码自动下载。

A.3.5 模型。我们在使用 Qwen1.5-1.8B 的设备上演示 llm.npu 的功能。用户还可以通过集成 Gemma-2B、Phi-2-2.7B、LLaMA-2-7B 和 Mistral-7B 等定制模型来执行其他评估。

A.4 安装

为了获得第 4.4 节中讨论的准确性结果，用户应该首先使用 Conda 安装在 precision_results/environment.yml 中指定的 Python 虚拟环境。该环境包含重现 llm.npu 准确性结果所需的所有依赖项。在评估基线方法时，特别是 SmoothQuant [84]，用户需要安装 SmoothQuant 环境，详细信息请参阅 precision_results/baselines/README.md。

对于第 4.2 节中概述的性能结果，建议用户参考并遵循 Performance_results 目录中每个相应 README.md 文件中提供的安装说明。

A.5 实验流程

最初，用户应该执行精度结果评估（accuracy_results），以生成使用 llm.npu 进行设备上推理所需的量化模型（accuracy_results/llm.npu/examples/act_scales_try/llmnp_get_int8_weights_finalmodel_int8bias_ns.py）。随后，使用生成的模型，用户应继续进行性能结果评估（performance_results）。

A.6 评价和预期结果

我们提供两种主要类型的评估结果：第 4.4 节的准确性评估结果和第 4.2 节的预填充性能评估结果。这些结果构成了表 6 中报告的数据和图 14 中所示的数据。

A.7 方法论

可用性、功能性和可重复性

参考

- [1] 人工智能核心。 <https://developer.android.com/ai/aicore>. 访问: [2023.7].
- [2] 苹果情报。 <https://www.apple.com/apple-intelligence/>. 访问: [2023.7].
- [3] Gboard - Google 键盘 - Google Play 上的应用程序 - play.google.com. <https://github.com/NVIDIA/FasterTransformer>. [访问日期: 2023 年 10 月 22 日].
- [4] Winogrande: 大规模的对抗性 winograd 模式挑战. 2019.
- [5] 一般数据保护法规。 <https://gdpr-info.eu/>, 2021.
- [6] AMD Strix Point (Ryzen 300). <https://www.sellcell.com/blog/how-most-do-people-upgrade-their-phone-2023-statistics>, 2023.
- [7] 提升 NPU. <https://www.hisilicon.com/en/products/Kirin/Kirin-flashing-chips/Kirin-9000>, 2023.
- [8] Edge TPU API. <https://coral.ai/docs/edge-tpu/inference/#general-Purpose-operating-systems>, 2023.
- [9] Gemma-2B. <https://huggingface.co/google/gemma-2b>, 2023.
- [10] HiAI 引擎。 <https://developer.huawei.com/consumer/cn/doc/hi-References/overview-0000001053824513>, 2023.
- [11] Llama-2-7b. <https://huggingface.co/mista-llama/Llama-2-7b-chat-hf>, 2023.
- [12] MediaTek APU 790. <https://corp.mediatek.com/news-events/press-releases/mediateks-new-all-big-core-design-for-flagship-dimensity-9300-chipset-maximizes-smartphone-performance-and-efficiency>, 2023.
- [13] 流星湖. <https://www.sellcell.com/blog/how-often-do-people-upgrade-their-phone-2023-statistics>, 2023.
- [14] Mistral-7B. <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>, 2023.
- [15] 神经飞行员. <https://neuropilot.mediatek.com/>, 2023.
- [16] Phi-2. <https://huggingface.co/microsoft/phi-2>, 2023.
- [17] Snapdragon 8gen3 SoC. <https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms/snapdragon-8-gen-3-mobile-platform>, 2023.
- [18] 人工智能基准. https://ai-benchmark.com/ranking_detailed.html, 2024.
- [19] Edgetpu. <https://cloud.google.com/edge-tpu>, 2024.
- [20] 基于 Gpt 的电子邮件编写器. <https://hix.ai/ai-email-writer-email-generator>, 2024.
- [21] 拥抱脸. <https://huggingface.co/>, 2024.
- [22] Llamatouch. <https://github.com/LlamaTouch/LlamaTouch>, 2024.
- [23] Mllm. <https://github.com/UbiquitousLearning/mllm>, 2024.
- [24] Mmlu 排行榜. <https://paperswithcode.com/sota/multi-task-language-understanding-on-mmlu>, 2024.
- [25] Qnn. <https://www.qualcomm.com/developer/software/qualcomm-ai-engine-direct-sdk>, 2024.
- [26] Marah Abidin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl 等. Phi-3 技术报告: 手机本地的高性能语言模型. arXiv 预印本 arXiv:2404.14219, 2024.
- [27] 白金泽、白帅、褚云飞、崔泽宇、党凯、邓晓东、杨帆、葛文斌、韩宇、黄飞、惠斌元、罗吉、李美、林俊阳、林润吉、刘大一恒、刘高、路成强、路克明、马建新、门锐、任行章、任宣成、谭传奇、谭思南、涂建宏、王鹏、王士杰、王伟、吴胜光、徐本峰、徐金、安阳、杨浩、杨建、杨树生、姚洋、于博文、袁宏毅、袁正、张建伟、张兴轩、张宜昌、张振如、周常、周静仁、周小欢和朱天航. Qwen 技术报告. arXiv 预印本 arXiv: 2309.16609, 2023.
- [28] Yushi Bai, Xin Lv, Jiajie Zhu, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, 等. Longbench: 用于长上下文理解的双语、多任务基准. arXiv 预印本 arXiv:2308.14508, 2023.
- [29] Daniel Bolya、付成阳、戴晓亮、张培照、Christoph Feichtenhofer 和 Judy Hoffman. 代币合并: 您的 vit, 但速度更快. arXiv 预印本 arXiv:2210.09461, 2022.
- [30] 蔡东奇, 吴耀宗, 王尚光, 林小竹, 徐孟伟. 现代自然语言处理的高效联邦学习. 第 29 届移动计算和网络国际会议论文集, 第 1-16 页, 2023 年.
- [31] Han Cai, Ji Lin, Yujun Lin, Zhijian Liu, Haotian Tang, Hanrui Wang, Ligeng Zhu and Song Han. 在移动设备上启用深度学习: 方法、系统和应用程序. ACM 电子系统设计自动化汇刊 (TODAES), 27(3):1-50, 2022.
- [32] Tianle Cai, Yuhong Li, Zhengyang Geng, hongwu Peng, Jason D Lee, Deming Chen 和 Tri Dao. Medusa: 具有多个解码头的简单 llm 推理加速框架. arXiv 预印本 arXiv: 2401.10774, 2024.
- [33] Tim Dettmers, Mike Lewis, Younes Belkada 和 Luke Zettlemoyer. Gpt3. int8(): Transformer 的 8 位矩阵乘法. 神经信息处理系统的进展, 35: 30318-30332, 2022.
- [34] Franz Franchetti 和 Markus Püschel. 生成 simd 矢量化排列. 国际编译器构建会议, 第 116-131 页. Springer, 2008.
- [35] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler 和 Dan Alishtarh. Gptq: 生成预训练 Transformer 的准确训练后量化. arXiv 预印本 arXiv:2210.17323, 2022.
- [36] Yichao Fu, Peter Bailis, Ion Stoica 和 Hao Zhang. 使用前瞻解码打破 llm 推理的顺序依赖性. arXiv 预印本 arXiv:2402.02057, 2024.
- [37] Petko Georgiev, Nicholas D Lane, Kiran K Rachuri 和 Cecilia Mascolo. DSP. Ear: 利用协处理器支持智能手机上的连续音频传感. 第 12 届 ACM 嵌入式网络传感器系统会议论文集, 第 295-309 页, 2014 年.
- [38] Huiguan, Shoshan Liu, Xiaolong Ma, Wei Niu, Bin Ren, Xipeng Shen, Yanzhi Wang 和 Pu Zhao. Cocopie: 通过压缩编译协同设计在现成的移动设备上启用实时人工智能. ACM 通讯, 64(6):62-68, 2021.
- [39] Donghee Ha, Mooseop Kim, KyeongDeok Moon 和 Chi Yoon Jeong. 通过统一内存上的分层处理器选择方法加速设备上学习. IEEE 传感器, 21(7):2364, 2021.
- [40] 韩立祥、周子木和李振江. Pantheon: 移动边缘 GPU 上的抢占式多 DNN 推理. 第 22 届移动系统、应用和服务国际会议论文集, 第 465-478 页, 2024 年.
- [41] Myeonggyun Han, Jihoon Hyun, Seongbeom Park, Jinsu Park 和 Woongki Baek. Mosaic: 异构、通信和约束感知模型切片和执行, 以实现准确高效的推理. 2019 年第 28 届并行架构和编译技术国际会议, 第 165-177 页. IEEE, 2019.
- [42] 何振宇, 钟泽轩, 蔡天乐, Jason D Lee, 何迪. 其余: 基于检索的推测解码. arXiv 预印本 arXiv: 2311.08252, 2023.
- [43] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song 和 Jacob Steinhardt. 测量大规模多任务语言理解. 国际学习表征会议 (ICLR) 会议记录, 2021 年.
- [44] Karla L Hoffman, Manfred Padberg, Giovanni Rinaldi 等人. 旅行商问题. 运筹学和管理科学百科全书, 1:1573-1578, 2013.
- [45] Kuan-Chieh Hsu 和 Hung-Wei Tseng. 同时和异构多线程. 第 56 届 IEEE/ACM 国际微架构研讨会论文集, 第 137-152 页, 2023 年.
- [46] Loc N Huynh, Youngki Lee 和 Rajesh Krishna Balan. Deepmon: 基于移动 GPU 的深度学习框架, 用于连续视觉应用. 第 15 届国际年会论文集

- 关于移动系统、应用程序和服务, 第 82-95 页, 2017 年。[47] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam 和 Dmitry Kalenichenko. 神经网络的量化和训练, 以实现高效的纯整数算术推理。IEEE 计算机视觉和模式识别会议论文集, 第 2704-2713 页, 2018 年。[48] Pegah Jandaghi, XiangHai Shen, Xinyi Bai, Jay Pujara 和 Hakim Sidahmed. 基于大型语言模型的基于角色的忠实会话数据集生成, 2023。[49] Xiaotang Jiang, Huan Wang, Yiliu Chen, Ziqi Wu, Lichuan Wang, Bin Zou, Yafeng Yang, Zongyang Cui, Yu Cai, Tianhang Yu, 等。Mnn: 通用且高效的推理引擎。arXiv 预印本 arXiv: 2002.12418, 2020。[50] Sehoon Kim, Kartikeya Mangalam, Jitendra Malik, Michael W Ma-honey, Amir Gholami 和 Kurt Keutzer. 大大小小的变压器解码器。arXiv 预印本 arXiv:2302.07863, 2023。[51] Sehoon Kim, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun 和 Kurt Keutzer. 学习了 Transformer 的 token 修剪。第 28 届 ACM SIGKDD 知识发现和数据挖掘会议记录, 第 784-794 页, 2022 年。[52] Youngsok Kim, Joonsung Kim, Dongju Chae, Daehyun Kim 和 Jang-woo Kim. μ 层: 使用协作单层加速和处理器友好的量化进行低延迟设备上推理。2019 年第十四届 EuroSys 会议记录, 第 1-15 页, 2019 年。[53] Nicholas D Lane, Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro 和 Fahim Kawsar. Deepix: 用于移动设备上低功耗深度学习推理的软件加速器。2016 年第 15 届 ACM/IEEE 传感器网络信息处理国际会议, 第 1-12 页。IEEE, 2016。[54] Royson Lee, Stylianos I Venieris, Lukasz Dudziak, Sourav Bhattacharya 和 Nicholas D Lane. Mobisr: 通过异构移动处理器实现高效的设备上超分辨率。第 25 届移动计算与网络国际会议, 第 1-16 页, 2019 年。[55] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jia Cheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 个人 llm 代理: 有关能力、效率和安全性的见解和调查。arXiv 预印本 arXiv: 2401.05459, 2024。[56] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang 和 Song Han. Awq: 用于 llm 压缩和加速的激活感知权重量化。arXiv 预印本 arXiv:2306.00978, 2023。[57] TensorFlow Lite. 在移动和物联网设备上部署机器学习模型, 2019。[58] Yi Liu, Yun Ma, Xusheng Shaw, Tao Xie, and Xuzhe Liu. Legodroid: 灵活的 Android 应用程序分解和即时安装。科学。中国信息网科学, 66(4), 2023。[59] llama.cpp. C/C++ 资源中 Facebook 的 LLaMA 模型的移植。https://github.com/ggerganov/llama.cpp, 出版年份。访问: [2023.7]。[60] 卢振彦, 李翔, 蔡东奇, 易荣杰, 刘方明, 张希文, 尼古拉斯·D·莱恩, 徐孟伟。小语言模型: 调查、测量和见解。arXiv 预印本 arXiv:2409.15790, 2024。[61] 苗旭鹏, Gabriele Oliaro, 张志浩, 程新浩, 王泽宇, Rae Ying Yee Wong, 陈卓明, Daiyaan Arfeen, Reyna Abhyankar 和志浩贾。Specinfer: 通过推测推理和令牌树验证加速生成 llm 服务。arXiv 预印本 arXiv:2305.09781, 2023。[62] Todor Mihaylov, Peter Clark, Tushar Khot 和 Ashish Sabharwal. 铠甲可以导电吗? 用于开卷问答的新数据集。EMNLP, 2018。[63] Wei Niu, Gagan Agrawal, Bin Ren. Sod2: 静态优化动态深度神经网络。arXiv 预印本 arXiv:2403.00176, 2024。[64] 牛伟, 管杰雄, 沉西鹏, 王彦志, Gagan Agrawal, 任斌。Gcd 2: 用于将 dnns 映射到移动 DSP 的全局优化编译器。2022 年第 55 届 IEEE/ACM 国际微架构研讨会 (MICRO), 第 512-529 页。IEEE, 2022。[65] 牛伟, 马小龙, 林盛, 王世豪, 钱学海, 林雪, 王彦之, 任斌。Patdnn: 通过基于模式的权重修剪在移动设备上实现实时 DNN 执行。《第二十五届编程语言和操作系统架构支持国际会议论文集》, 第 907-922 页, 2020 年。[66] Wei Niu, Md Musfiqur Rahman Sanim, Zhihao Shu, Jiexiongguan, Xipeng Shen, Miao Yin, Gagan Agrawal 和 Bin Ren. Smartmem: 布局转换消除和适应, 以在移动设备上高效执行 DNN。第 29 届 ACM 编程语言和操作系统架构支持国际会议论文集, 第 3 卷, 第 916-931 页, 2024 年。[67] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh 和 Jordan Soyke. Tensorflow-serving: 灵活、高性能的机器学习服务。arXiv 预印本 arXiv: 1712.06139, 2017。[68] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda 和 Raquel Fernandez. LAMBADA 数据集: 需要广泛的语篇上下文的单词预测。计算语言学协会第 54 届年会议论文集 (第一卷: 长论文), 第 1525-1534 页, 德国柏林, 2016 年 8 月。计算语言学协会。[69] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki 和 Ricardo Bianchini。Splitwise: 使用相分裂进行高效的生成 llm 推理。arXiv 预印本 arXiv:2311.18677, 2023。[70] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal 和 Jeff Dean. 有效扩展变压器推理。机器学习和系统论文集, 2023 年 5 月。[71] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand 和 Saman Amarasinghe. Halide: 一种用于优化图像处理管道中的并行性、局部性和重新计算的语言和编译器。Acm Sigplan 公告, 48(6):519–530, 2013。[72] 饶永明、赵文亮、刘本林、陆继文、周杰和谢卓瑞。Dynamicvit: 具有动态令牌稀疏化的高效视觉转换器。神经信息处理系统的进展, 34: 13937–13949, 2021。[73] Alexander J Root, Maaz Bin Saifeer Ahmad, Dillon Sharlet, Andrew Adams, Shoaib Kamil 和 Jonathan Ragan-Kelley. 用于快速数字信号处理的快速指令选择。第 28 届 ACM 编程语言和操作系统架构支持国际会议论文集, 第 4 卷, 第 125-137 页, 2023 年。[74] Yixin Song, Zeyu Mi, Haotong Xie 和 Haibo Chen. Powerinfer: 使用消费级 GPU 提供服务的快速大型语言模型。arXiv 预印本 arXiv:2312.12456, 2023。[75] MLC 团队。MLC-法学硕士。https://github.com/mlc-ai/mlc-llm, 2023。[76] 塞缪尔·托马斯和詹姆斯·博恩霍尔特。自动生成用于可定制数字信号处理器的矢量化编译器。2024。[77] 菲利普·蒂莱, 孔祥宗, 大卫·考克斯。Triton: 一种用于平铺神经网络计算的中间语言和编译器。第三届 ACM SIGPLAN 国际机器学习和编程语言研讨会论文集, 第 10-19 页, 2019 年。[78] Sander Vocke, Henk Corporaal, Roel Jordans, Rosilde Corvino 和 Rick Nas. 扩展 halide 以改进成像 DSP 的软件开发。ACM 架构和代码优化交易 (TACO), 14(3):1–25, 2017。

[79] 王涵瑞, 张哲凯, 韩松。Spatten: 具有级联令牌和头部修剪的高效稀疏注意力架构。2021 年 IEEE 国际高性能计算机架构研讨会 (HPCA), 第 97-110 页。IEEE, 2021。[80] 王文辉, 魏福如, 董立, 鲍航波, 杨南, 周明。Minilm: 深度自注意力蒸馏, 用于预训练 Transformer 的任务无关压缩。神经信息处理系统进展, 33: 5776–5788, 2020。[81] 文浩, 李元春, 刘国宏, 赵山辉, 于涛, 李家军, 蒋士奇, 刘云浩, 张亚勤, 刘云欣。使大语言模型使用智能手机实现智能任务自动化。arXiv 预印本 arXiv:2308.15272, 2023。[82] 文浩, 李元春, 刘国宏, 赵山辉, 余涛, 李家军, 蒋士奇, 刘云浩, 张亚勤和刘云欣。Autodroid: Android 中由 Llm 驱动的任务自动化。第 30 届移动计算和网络国际会议论文集, 第 543-557 页, 2024 年。[83] Bingyang Wu, Shengyu Liu, Yinminzhong, Peng Sun, Xuanzhe Liu 和 Xin Jin。Loongserve: 通过弹性序列并行性有效地服务长上下文大型语言模型。arXiv 预印本 arXiv:2404.09526, 2024。[84] Guangxuan Shaw, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth 和 Song Han。Smoothquant: 大型语言模型的准确高效的训练后量化。国际机器学习会议, 第 38087-38099 页。PMLR, 2023。[85] 徐大良, 李庆, 徐孟伟, 黄康, 黄刚, 王尚光, 金鑫, 马云, 刘轩哲。Niagara: 在异构边缘处理器上调度 dnn 推理服务。面向服务计算国际会议, 第 67-85 页。Springer, 2023。[86] 徐大亮, 徐孟伟, 王启鹏, 王尚光, 马云, 黄康, 黄刚, 金鑫, 刘轩哲。Mandheling: 具有 DSP 卸载的混合精度设备上 DNN 训练。第 28 届移动计算与网络国际会议论文集, 第 214-227 页, 2022 年。[87] Dailiang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Meng-wei Xu 和 Xuanzhe Liu。Llmcad: 快速且可扩展的设备上大型语言模型推理。arXiv 预印本 arXiv:2309.04255, 2023。[88] 徐大亮, 张浩, 杨黎明, 刘瑞琪, 徐孟伟, 刘轩哲。Wip: 使用移动 NPU 进行高效 llm 预填充。边缘和移动基础模型研讨会论文集, EdgeFM '24, 第 33-35 页, 美国纽约州纽约市, 2024 年。计算机协会。[89] 徐孟伟, 蔡东奇, 吴耀宗, 李翔, 王尚光。{FwdLLM}: 通过扰动推理对大型语言模型进行高效联合微调。2024 年 USENIX 年度技术会议 (USENIX ATC 24), 第 579-596 页, 2024 年。[90] Mengwei Xu, Jiawei Liu, Yuanqiang Liu, Felix Xiaozhu Lin, Yunxin Liu 和 Xuanzhe Liu。初探智能手机上的深度学习应用程序。万维网会议, 第 2125-2136 页, 2019 年。[91] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Dailiang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhu, Chen Yang, Shihe Wang, et al。对资源高效的 LLM 和多模式基础模型的调查。arXiv 预印本 arXiv:2401.08092, 2024。[92] Mengwei Xu, Mengze Zhu, Yunxin Liu, Felix Xiaozhu Lin 和 Xuanzhe Liu。Deepcache: 用于移动深度视觉的原则性缓存。在

第 24 届移动计算与网络国际会议论文集, 第 129-144 页, 2018 年。[93] 刘轩哲, 赵一豪, 刘树凡, 李翔, 朱鑫, 刘一波, 金鑫。Muxflow: 在超过 10,000 个 GPU 的生产级集群中高效共享 GPU。中国科学信息科学, 页数 -, 2024。[94] 薛振良, 宋一欣, 米泽宇, 陈乐, 夏玉斌, 陈海波。Powerinfer-2: 智能手机上的快速大型语言模型推理。arXiv 预印本 arXiv:2406.06282, 2024。[95] Seongjun Yang, Gibbeum Lee, Jaewoong Cho, Dimitris Papailiopoulos 和 Kangwook Lee。预测流水线解码: 精确 llm 解码的计算延迟权衡。arXiv 预印本 arXiv:2307.05908, 2023。[96] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li 和 Yuxiong He。Zeroquant: 针对大型 Transformer 的高效且经济实惠的训练后量化。神经信息处理系统的进展, 35: 27168–27183, 2022。[97] Juheon Yi 和 Youngki Lee。Heimdall: 用于增强现实应用的移动 GPU 协调平台。第 26 届移动计算与网络国际会议论文集, 第 1-14 页, 2020 年。[98] Rongjie Yi, LiweiGuo, Shiyun Wei, Ao Zhou, Shangguang Wang 和 Mengwei Xu。Edgemoe: 基于 moe 的大型语言模型的快速设备上推理。arXiv 预印本 arXiv:2308.14352, 2023。[99] Wangsong Yin, Mengwei Xu, Yuanchun Li, Xuanzhe Liu。Llm 作为移动设备上的系统服务。arXiv 预印本 arXiv:2403.11805, 2024。[100] Wangsong Yin, Rongjie Yi, Dailiang Xu, Gang Huang, Mengwei Xu, Xuanzhe Liu。Elms: 移动设备上的弹性大型语言模型。arXiv 预印本 arXiv:2409.09071, 2024。[101] 赵友, 冯书林, 苏丹, 余东。Speechmoe2: 具有改进路由的专家混合模型。ICASSP 2022-2022 IEEE 国际声学、语音和信号处理会议 (ICASSP), 第 7217-7221 页。IEEE, 2022。[102] 袁金良, 杨晨, 蔡东奇, 王世和, 袁鑫, 张泽岭, 李翔, 张丁格, 梅涵子, 贾贤庆, 等。作为固件的移动基础模型。第 30 届移动计算和网络国际会议论文集, 第 279-295 页, 2024 年。[103] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi 和 Yejin Choi。Hellawag: 机器真的能完成你的句子吗? 计算语言学协会第 57 届年会论文集, 2019 年。[104] 曾群松, 杜雨清, 黄凯斌和梁健 K。通过 CPU-GPU 异构计算进行联合边缘学习的节能资源管理。IEEE 无线通信汇刊, 20(12): 7947–7962, 2021。[105] Jinrui Zhang, Deyu Zhang, Xiaohui Xu, Fu Cheng Jia, Yunxin Liu, Xuanzhe Liu, Ju Ren 和 Yaoxue Zhu。Mobipose: 移动设备上的实时多人姿势估计。第十八届嵌入式网络传感器系统会议论文集, 第 136-149 页, 2020 年。[106] Yinmin Zhu, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin 和 Hao Zhang。Distserve: 分解预填充和解码, 以提供优化的大型语言模型服务。arXiv 预印本 arXiv:2401.09670, 2024。